



# Cortextrace

Local-first observability and threat detection for AI agents

First public preview · Version 0.1.0 · 17 September 2026

---

COMPANY

AI and ML consultants

MAINTAINER

Manjunatha M

WEBSITE

<https://aiandmlconsultants.com/>

REPOSITORY

[github.com/manjunatha552355-rgb/cortextrace](https://github.com/manjunatha552355-rgb/cortextrace)

CONTACT

[Manjunatha@aiandmlconsultants.com](mailto:Manjunatha@aiandmlconsultants.com)

LICENSE

Apache License 2.0

# Contents

|    |                                      |    |
|----|--------------------------------------|----|
| 02 | Executive Introduction               | 2  |
| 03 | The Problem                          | 4  |
| 04 | The Solution                         | 5  |
| 05 | Product Overview                     | 6  |
| 06 | Agent Discovery                      | 8  |
| 07 | Multi-Agent Observability            | 12 |
| 08 | Real-Time Activity Monitoring        | 14 |
| 09 | Session Tracing                      | 21 |
| 10 | Security Intelligence                | 25 |
| 11 | Agent Risk Analysis                  | 30 |
| 12 | Interactive Security Dashboard       | 31 |
| 13 | Agent Relationship Graph             | 33 |
| 14 | Analytics                            | 35 |
| 15 | Privacy and Security Architecture    | 38 |
| 16 | Technical Architecture               | 41 |
| 17 | Extensibility                        | 44 |
| 18 | Installation and Getting Started     | 46 |
| 19 | Supported Platforms and Capabilities | 50 |
| 20 | Security and Threat Model            | 51 |
| 21 | Product Release Summary              | 53 |
| 22 | Company and Contact                  | 55 |

## Document information

|                   |                                           |
|-------------------|-------------------------------------------|
| Product           | Cortextrace 0.1.0                         |
| Release           | First public preview, 17 September 2026   |
| Document revision | 1.0                                       |
| Author            | Manjunatha M, AI and ML consultants       |
| License           | Apache License 2.0 (open source)          |
| Distribution      | Public — prepared for open-source release |

## 02 Executive Introduction

Cortextrace is a desktop application for Windows, macOS and Linux that discovers the AI agents running on a computer, records what they do, and flags behaviour that looks risky, with an explanation of why.

It belongs to an emerging category that combines **AI-agent observability** (what did the agent do, in which session, in what order) with **endpoint security monitoring** (did anything it did look dangerous). Cortextrace runs entirely on the user's machine: data is stored locally, nothing is sent anywhere by default, and no account is needed.

### Value proposition

Developers and security teams increasingly let AI coding agents run shell commands, edit files and call external tools. Cortextrace gives them one place to see which agents are active, reconstruct every session as a timeline, and review explainable detections, without sending prompts or source code to a third party.

### About the screenshots in this document

All screenshots were captured from the actual Cortextrace 0.1.0 application. To avoid exposing personal data, the application was run with an isolated demonstration profile. Agent activity, detections, alerts and risk scores shown were produced by the application's own engine from **synthetic telemetry** generated with the bundled generator and sent through the real hook, OpenTelemetry and API endpoints. Risk scores next to real product names (for example Claude Code or Cursor) reflect these injected demonstration scenarios and say nothing about the behaviour of those products. The process tree view shows a harmless demonstration agent process; the agent inventory also lists agents that were genuinely running on the capture workstation.

## Primary users

| User                               | What they use Cortextrace for                                                                                                               |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Developers using AI coding agents  | See exactly which commands, files and MCP tools their agents touched; catch unintended destructive or credential-related actions.           |
| Security and platform engineers    | Review detections with evidence, define policies (including approval requirements for supported agents), export events to existing tooling. |
| Engineering leaders and architects | Understand agent adoption and behaviour across repositories through analytics, without a cloud service.                                     |
| Agent and tool builders            | Send structured telemetry from custom agents through the local API or OpenTelemetry and inspect it in the dashboard.                        |

## This release at a glance

| Item                 | Value                                                                                                                        |
|----------------------|------------------------------------------------------------------------------------------------------------------------------|
| Product              | Cortextrace                                                                                                                  |
| Version              | 0.1.0 — first public preview                                                                                                 |
| Release date         | 17 September 2026 (project changelog)                                                                                        |
| License              | Apache License 2.0                                                                                                           |
| Platforms            | Windows, macOS, Linux (see section 19 for validation status per platform)                                                    |
| Maintainer           | Manjunatha M, AI and ML consultants                                                                                          |
| Source and downloads | <a href="https://github.com/manjunatha552355-rgb/cortextrace">https://github.com/manjunatha552355-rgb/cortextrace</a>        |
| Distribution         | Windows NSIS installer and MSI (built and install-tested); macOS DMG/PKG and Linux AppImage/DEB/RPM build targets configured |

## 03 The Problem

AI agents have moved from suggesting code to acting on the developer's machine. The tooling to see what they do has not kept up.

### Agents now act, not just suggest

Modern coding agents run shell commands, install packages, read and write files across a repository, call MCP servers and reach network services on the user's behalf. Many developers run several of them at once: a terminal agent, an IDE assistant, a desktop assistant and sometimes locally built agents.

### Visibility gaps

| Gap                               | Consequence                                                                                                                                                                 |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| No unified inventory              | It is hard to answer a basic question: which AI agents are installed and running on this machine right now?                                                                 |
| Fragmented, per-product telemetry | Each agent reports (if at all) in its own format — hooks, OpenTelemetry, log files — with no common session model.                                                          |
| Tool and command opacity          | A long agent session can execute dozens of commands and file edits; reconstructing what happened afterwards is manual.                                                      |
| Concurrent agents                 | When several agents work at once, activity from different sessions interleaves and is difficult to attribute.                                                               |
| Security review without context   | A risky command, a read of a credential file or a prompt-injection string in tool output may go unnoticed, or be noticed without the surrounding sequence that explains it. |
| Privacy tension                   | Sending prompts and source code to an external monitoring service to gain visibility creates a new exposure.                                                                |

### Why it matters

- Agents operate with the user's permissions: whatever the user can delete, read or upload, an agent can too.
- Untrusted content (web pages, issue text, MCP tool responses) can carry instructions aimed at the model, so an agent can be steered into unintended actions.
- Teams adopting agents need evidence — not guesses — to set sensible guardrails and to investigate when something goes wrong.

# 04 The Solution

Cortextrace treats every AI agent as an observable workload on the endpoint: it discovers agents, normalises their activity into one event model, analyses each event as it arrives, and presents the result in a desktop dashboard.

| Principle               | How Cortextrace implements it                                                                                                                                                   |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Agent discovery         | Process signatures, known installation locations, runtime-reported identity (hooks, OpenTelemetry) and user-defined custom agents.                                              |
| Unified observability   | One versioned event schema (27 event types) for hooks, OpenTelemetry, the REST API and OS-level collectors, with sessions, correlation ids and trace ids.                       |
| Security monitoring     | Layered detection on every event: 39 tested rules, user policies, stateful heuristics and learned activity baselines, with explicit classification of findings.                 |
| Explainable risk        | Risk scores per session and per agent across 10 dimensions, each showing its contributing factors and evidence.                                                                 |
| Real-time visualisation | Hook, OpenTelemetry and API events are processed in batches of at most 200 ms and streamed to the dashboard; OS collectors poll on fixed intervals (process 5 s, network 10 s). |
| Local-first             | SQLite storage in the user profile, loopback-only API, no outbound traffic by default, prompts stored as metadata by default.                                                   |

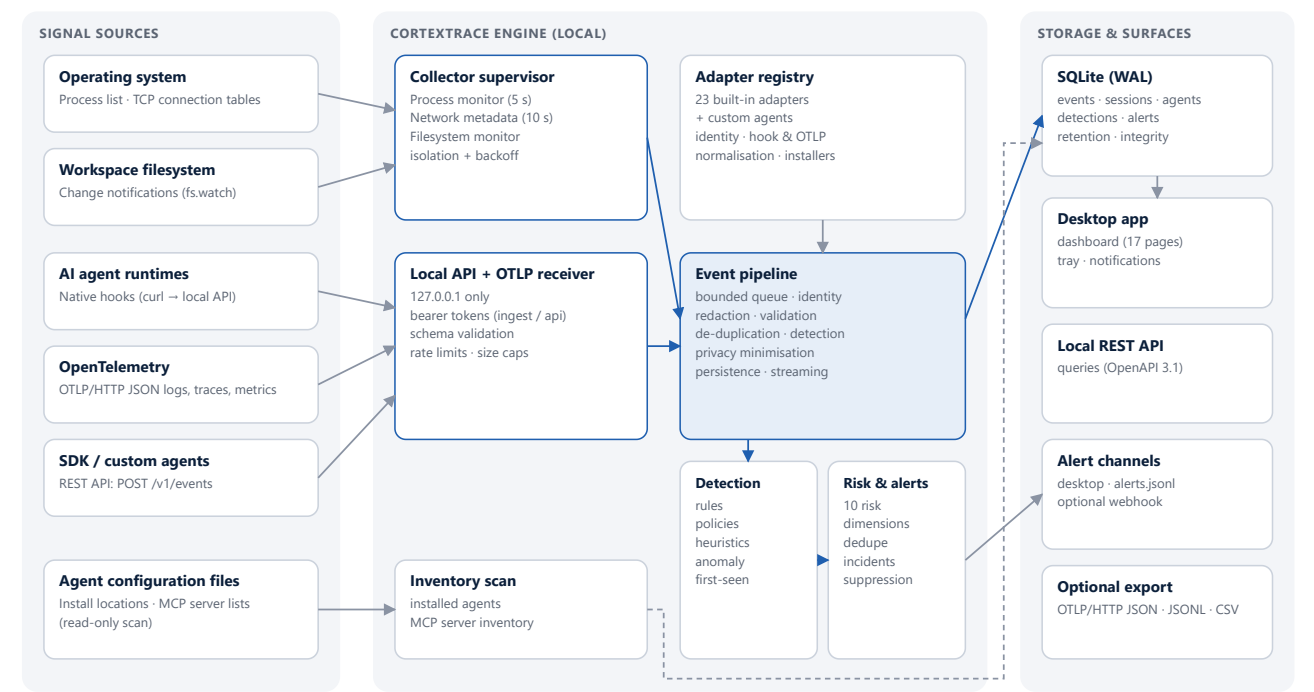
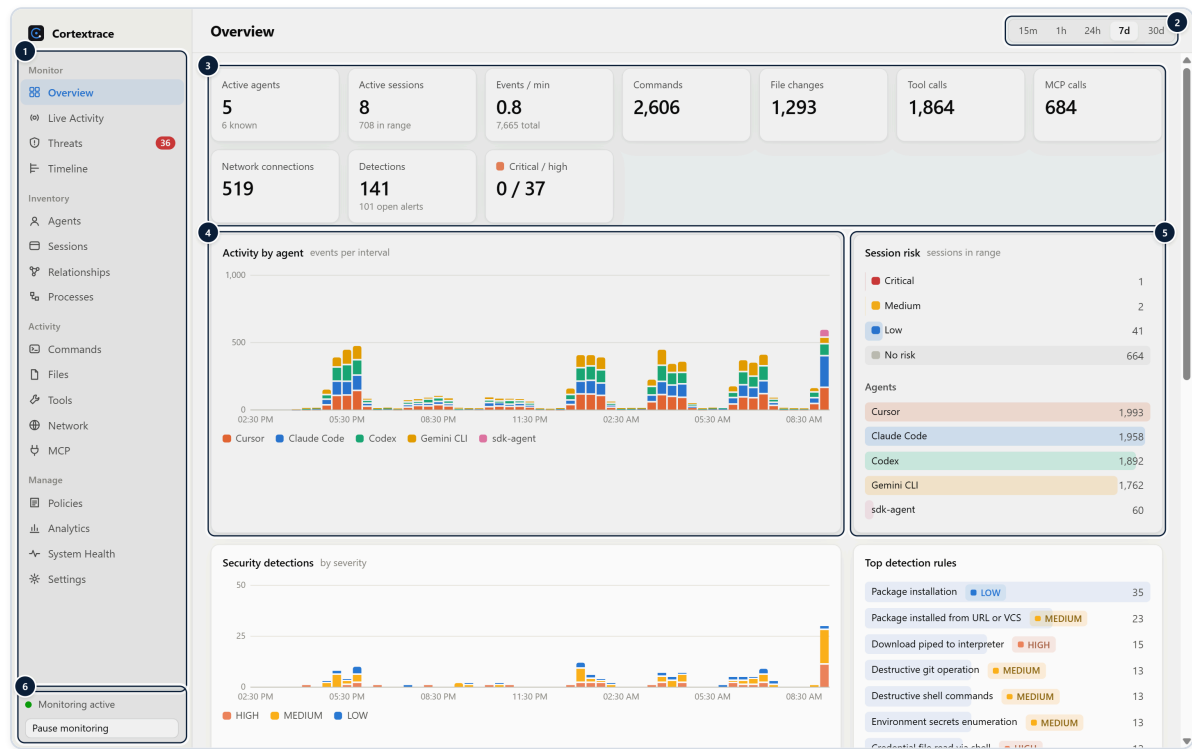


Figure 1 High-level product architecture: signal sources, the local engine, and storage and user-facing surfaces.

# 05 Product Overview

Cortextrace is a single desktop application. Its engine runs inside the application process (or headless, from the command line) and serves a dashboard organised into four areas: Monitor, Inventory, Activity and Manage.



**Figure 2** The Overview dashboard summarises agents, sessions, activity and security posture for the selected time range.

- 1

**Navigation** — Monitor, Inventory, Activity and Manage sections; the Threats badge shows open alerts from the last 24 hours.
- 2

**Time range** — Global time range (15 minutes to 30 days) applied to every chart and table.
- 3

**KPIs** — Active agents, sessions, event rate, commands, file changes, tool/MCP calls, network connections and detections. Each tile links to its detail page.
- 4

**Activity by agent** — Stacked event counts per interval, one colour per agent; hover for values, click a column to open that window in the Timeline.
- 5

**Risk & agent distribution** — Sessions grouped by risk level and events per agent for the selected range.
- 6

**Monitoring state** — Current monitoring state and one-click pause/resume.

## Major modules

| Module           | Responsibility                                                                                                                  | Source                       |
|------------------|---------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| Collectors       | Process tree snapshots, per-process network connection metadata, workspace file watching; supervised with isolation and backoff | <code>src/collectors/</code> |
| Adapter registry | 23 built-in agent adapters and custom agents: identification, hook and OTLP normalisation, integration installers               | <code>src/adapters/</code>   |

| Module                          | Responsibility                                                                                             | Source                                                 |
|---------------------------------|------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Event pipeline                  | Queueing, identity, redaction, validation, de-duplication, detection, minimisation, persistence, streaming | <code>src/core/engine.ts</code>                        |
| Detection, policy, risk, alerts | Rules, policies, heuristics, anomaly detection, risk scoring, alert de-duplication and incidents           | <code>src/detection/</code>                            |
| Storage                         | SQLite database with migrations, indexes, retention, integrity checks and export                           | <code>src/storage/db.ts</code>                         |
| Local API                       | Token-authenticated REST API, OTLP/HTTP JSON receiver, OpenAPI 3.1 document                                | <code>src/api/</code>                                  |
| Desktop shell and dashboard     | Electron main process (tray, notifications, IPC) and React dashboard                                       | <code>src/main/</code> ,<br><code>src/renderer/</code> |

## User journey

1. **Install and launch.** The first-run guide discloses every data source and lists agents found on the machine.
2. **Connect agents (optional).** One click installs native hooks or OpenTelemetry settings for Claude Code, Cursor, Codex and Gemini CLI, after backing up the agent's configuration.
3. **Work normally.** Cortextrace records agent sessions in the background, including when the window is closed to the tray.
4. **Review.** Live Activity, Sessions and the Timeline show what happened; Threats shows detections grouped into incidents; desktop notifications surface high-severity alerts.
5. **Tune.** Policies, allowlists, rule toggles, suppressions and privacy modes are adjusted in Policies and Settings.

## 06 Agent Discovery

### Agent discovery and identification

#### WHAT IT IS

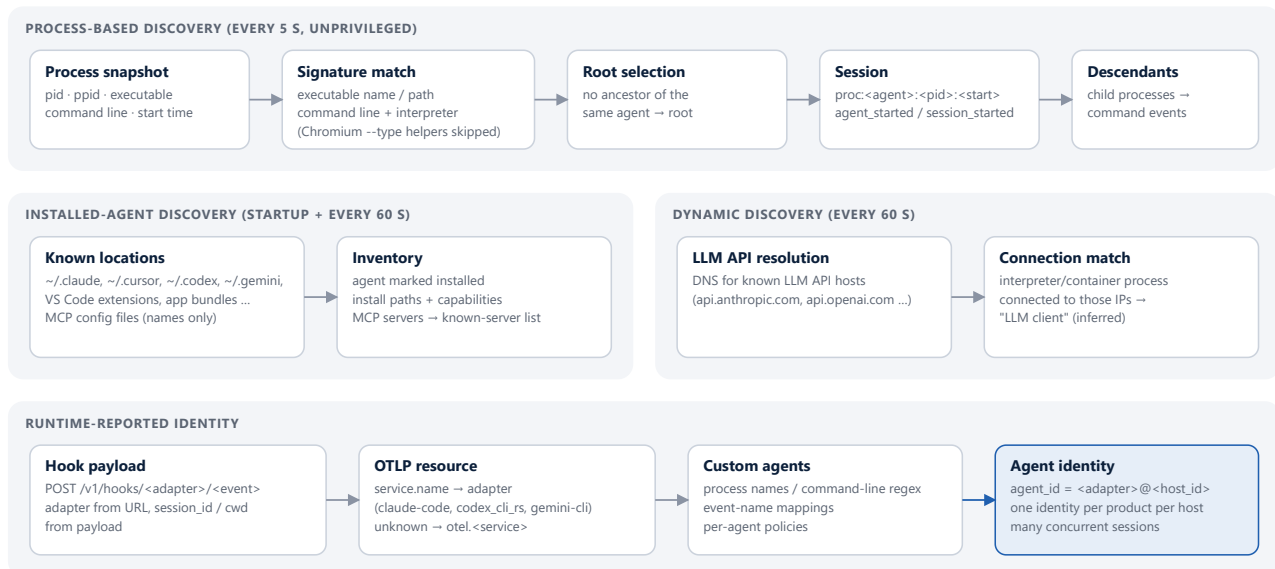
Automatic detection of AI agents that are installed or running on the machine, and assignment of a stable identity to each.

#### WHY IT MATTERS

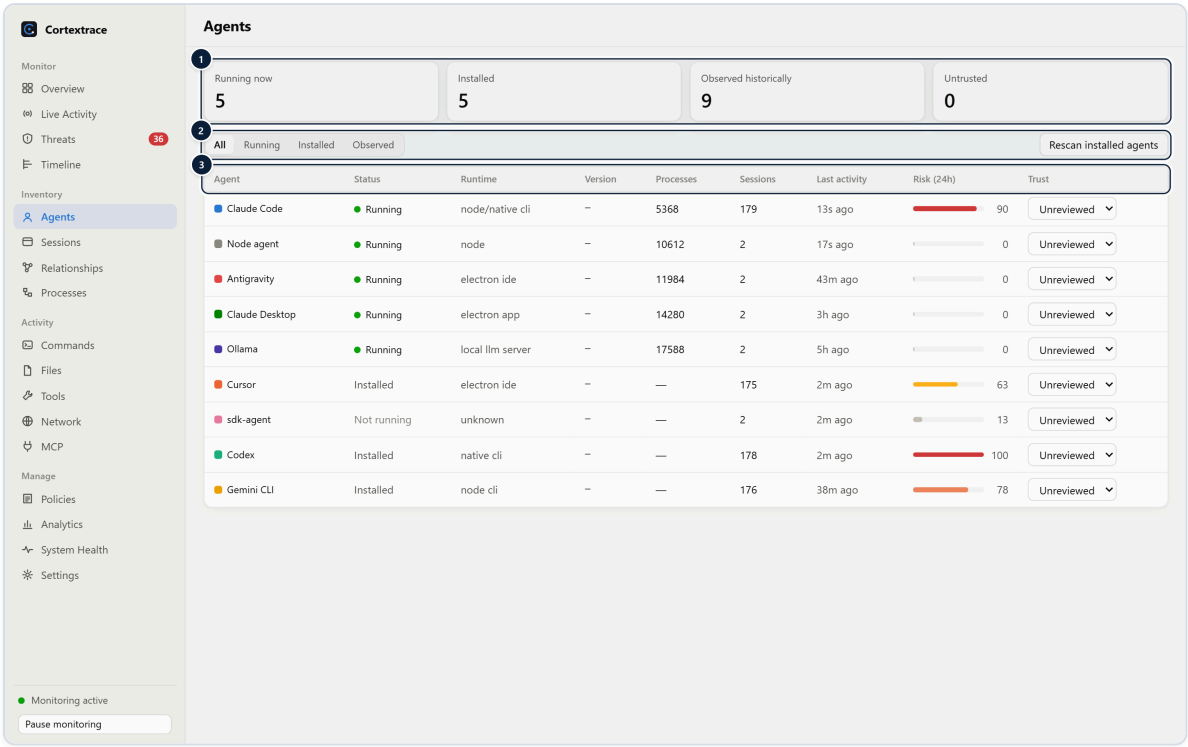
An inventory is the starting point for any review: users cannot assess agents they do not know are present.

#### HOW IT WORKS

- **Process signatures.** Every 5 seconds the process collector reads the OS process list (PowerShell/CIM on Windows, `/proc` on Linux, `ps` on macOS) and matches executable names, paths and command lines against adapter signatures. Chromium/Electron helper processes are ignored as roots.
- **Installed agents.** At startup and every 60 seconds, known installation and configuration locations are checked (for example `~/claude`, `~/cursor`, `~/codex`, `~/gemini`, VS Code extension folders, application bundles).
- **Runtime-reported identity.** Hook requests name their adapter in the URL; OpenTelemetry data is attributed through the resource `service.name`.
- **Dynamic discovery.** Interpreter or container processes connected to known LLM API endpoints are recorded as an "LLM client" with `inferred` fidelity.
- **Identity.** An agent is identified as `<adapter>@<host_id>`; each running instance or reported conversation becomes a separate session.



**Figure 3** Agent discovery flow: process signatures, installed-agent scan, dynamic discovery and runtime-reported identity.



**Figure 4** Agent inventory: running, installed and historically observed agents with process ids, sessions, 24-hour risk and trust status.

- 1

**Inventory summary** — Running, installed, historically observed and untrusted agent counts.
- 2

**Filter & rescan** — Filter by running/installed/observed; rescan known installation locations.
- 3

**Agents** — Status, runtime, version, live process ids, sessions, last activity, 24-hour risk and trust (unreviewed / trusted / untrusted).

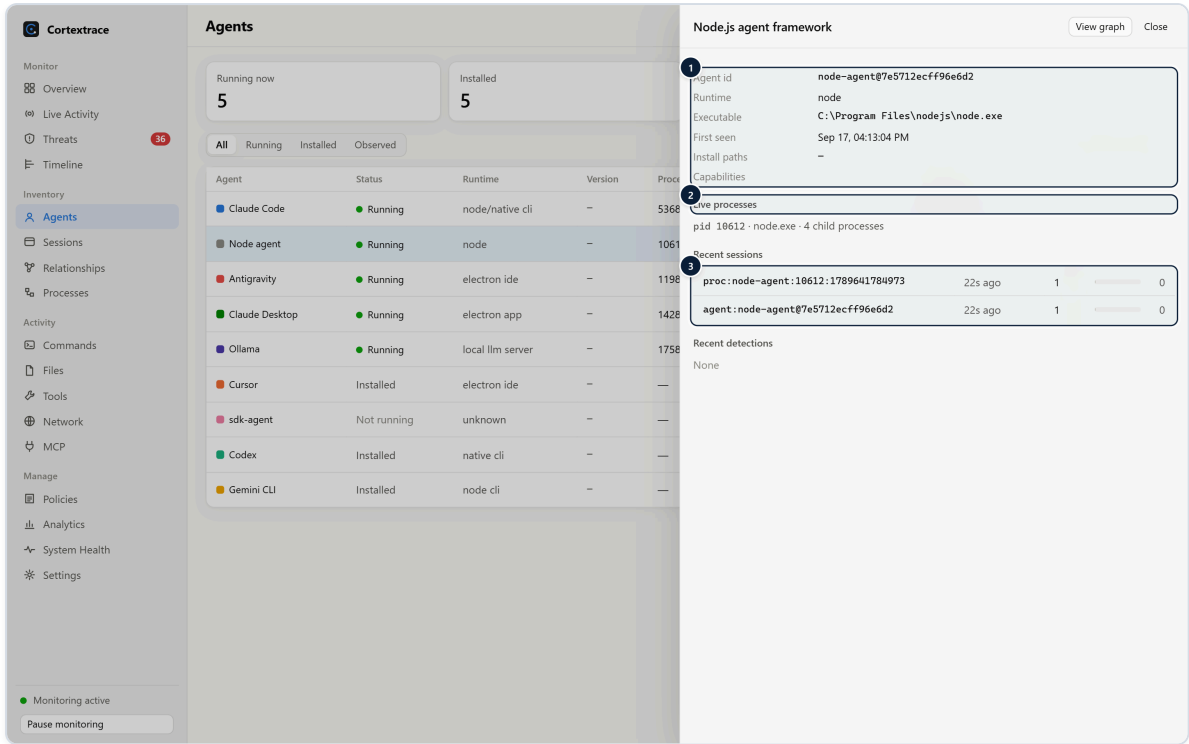


Figure 5 Agent detail drawer for a process-discovered agent.

- 1

**Identity** — Agent id, runtime, executable, install paths and declared adapter capabilities.
- 2

**Live processes** — Current root processes with child counts.
- 3

**Sessions** — Recent sessions with activity and risk; click to open the session timeline.

Supported agents

The following adapters are built in. "Installer" means Cortextracex can write the agent's native hook or telemetry configuration for the user; all other agents are discovered from processes and installation locations, and can additionally send telemetry through the generic OTLP receiver or REST API.

| Agent                                        | Discovery                              | Native integration                                                 |
|----------------------------------------------|----------------------------------------|--------------------------------------------------------------------|
| Claude Code                                  | process · install location             | Hooks (installer) · OpenTelemetry (installer) · approval responses |
| Codex                                        | process · install location             | Hooks (installer) · OpenTelemetry (installer) · approval responses |
| Cursor                                       | process · install location             | Hooks (installer) · approval responses on before-hooks             |
| Gemini CLI                                   | process · install location             | OpenTelemetry (installer)                                          |
| Claude Desktop / Cowork                      | process · install location             | MCP configuration inventory                                        |
| GitHub Copilot (VS Code), GitHub Copilot CLI | process · extension / install location | Generic OTLP receiver (CLI)                                        |
| Antigravity, Windsurf, Kiro, ChatGPT Desktop | process · install location             | —                                                                  |

| Agent                               | Discovery                                       | Native integration                                        |
|-------------------------------------|-------------------------------------------------|-----------------------------------------------------------|
| Cline, Roo Code                     | process (extension host) · extension location   | —                                                         |
| OpenCode, Aider, goose, Amp         | process · install location                      | Generic OTLP receiver where the agent supports it         |
| Ollama, LM Studio, llama.cpp        | process · install location                      | — (local model servers)                                   |
| MCP server processes                | process command line                            | —                                                         |
| Python and Node.js agent frameworks | process command line (framework names)          | REST API / OTLP                                           |
| Custom agents                       | user-defined process names / command-line regex | Hook endpoint with event-name mapping; per-agent policies |

**Platform considerations**

Process discovery requires no elevated privileges. Processes owned by other users may show partial metadata. On Windows, the working directory of another process is not available without debugging privileges, so process-only sessions on Windows have no workspace; hook and OpenTelemetry integrations provide it.

# 07 Multi-Agent Observability

## Concurrent agent sessions

### WHAT IT IS

Simultaneous tracking of any number of agents and sessions, each isolated by session id and attributed to an agent identity.

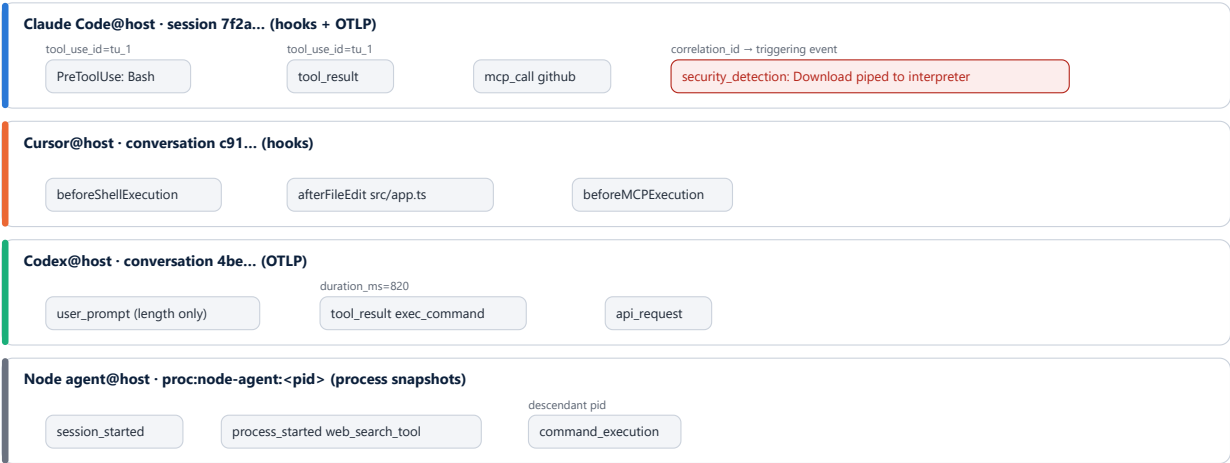
### WHY IT MATTERS

Developers commonly run several agents at the same time; without session isolation their activity cannot be separated or explained.

### HOW IT WORKS

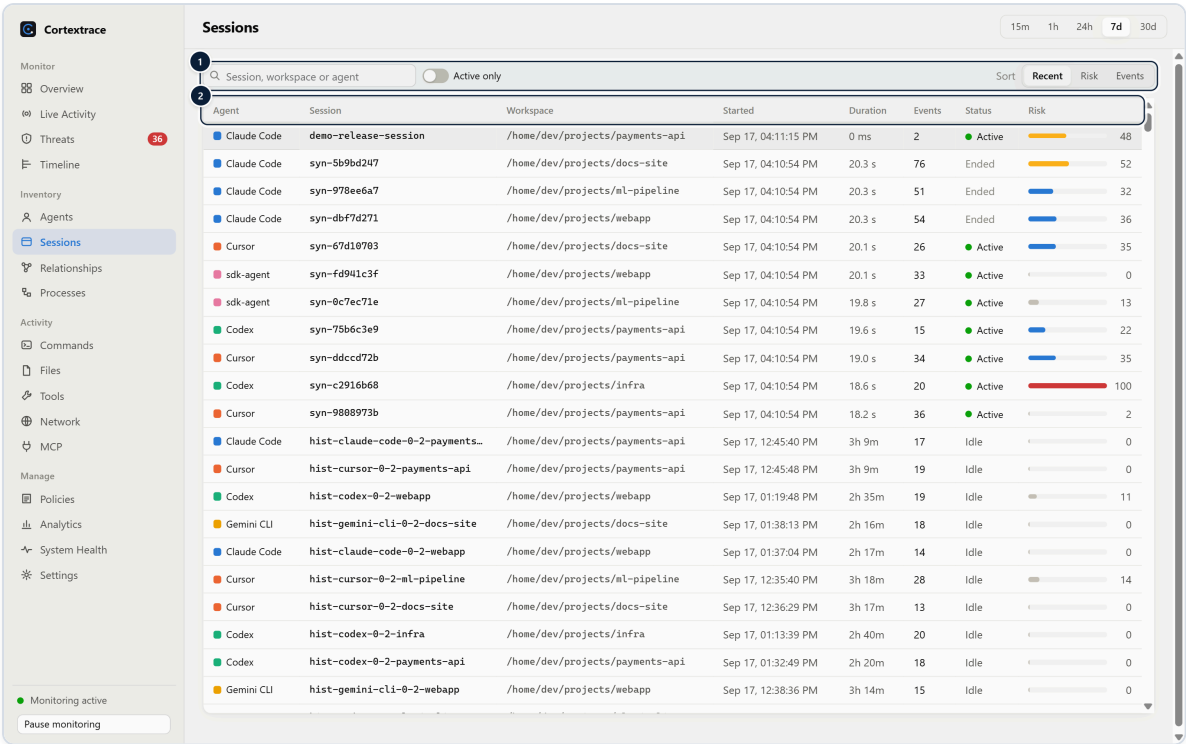
- Every event carries `agent_id`, `agent_type` and `session_id`. Sessions come from the runtime (Claude Code `session_id`, Cursor `conversation_id`, OTLP `session.id` / `conversation.id`) or from process roots (`proc:<agent>:<pid>:<start>`).
- Tool calls and their results are linked by `correlation_id`; detection events reference the triggering event; OpenTelemetry trace and span ids are preserved.
- The same action reported by a hook, an OpenTelemetry log and a process snapshot is de-duplicated before storage.
- Nested agents (for example an MCP server started by an agent) are tracked as their own agent and do not inherit the parent's descendants.

Concurrent sessions are isolated by session\_id and attributed to an agent identity; events inside a session are linked by correlation\_id and OpenTelemetry trace/span ids.



Many instances of one product → one agent\_id, many session\_ids. Hook, OTLP and process reports of the same action are de-duplicated before storage.

Figure 6 Multi-agent tracing: four concurrent sessions from different agents and collection methods.



**Figure 7** Sessions list across all agents with duration, event counts, status and risk.

- 1

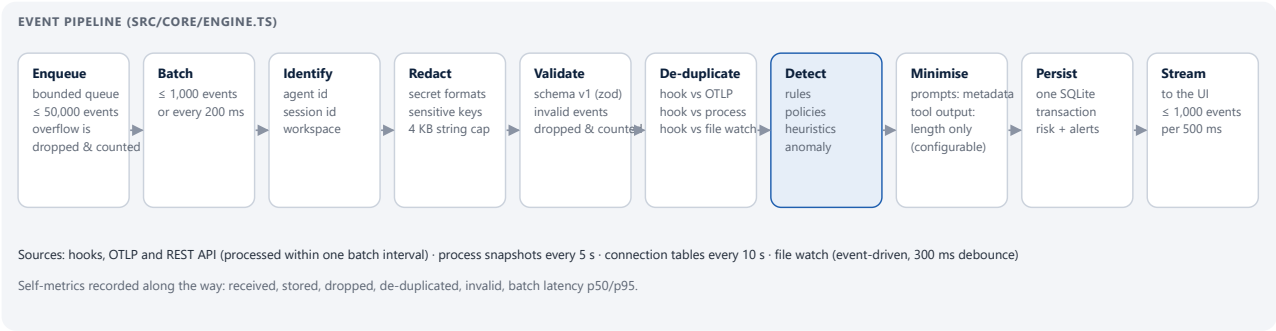
**Filter & sort** — Search by session, workspace or agent; active-only filter; sort by recency, risk or event count.
- 2

**Session inventory** — Agent, session id, workspace, start, duration, event count, status and risk score per session.

The benchmark suite ingests activity from 150 concurrent sessions; the demonstration dataset used for the screenshots contains 718 sessions across six agent types.

## 08 Real-Time Activity Monitoring

Cortextrace normalises every observation into one event model and streams it to the dashboard.



**Figure 8** Event ingestion pipeline with bounded queueing, redaction, validation, de-duplication, detection and minimisation.

| Source                                           | Latency characteristics                        | What it provides                                                                                              |
|--------------------------------------------------|------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Agent hooks (Claude Code, Cursor, Codex, custom) | Processed within one batch interval (≤ 200 ms) | Tool calls, commands, file operations, MCP calls, prompts (metadata by default), approvals, session start/end |
| OpenTelemetry (OTLP/HTTP JSON)                   | Processed within one batch interval            | Tool results with durations, prompts (length), API requests, errors, approval decisions, spans                |
| REST API ( <code>POST /v1/events</code> )        | Processed within one batch interval            | Any of the 27 event types from custom agents and SDKs                                                         |
| Process collector                                | Polling, default every 5 s                     | Agent start/stop, session start/end, child processes (commands)                                               |
| Network collector                                | Polling, default every 10 s                    | Remote IP, port and resolved LLM API hostname per agent process — connection metadata only                    |
| Filesystem collector                             | Event-driven, 300 ms debounce                  | Created / modified / deleted paths in active agent workspaces (attribution inferred)                          |

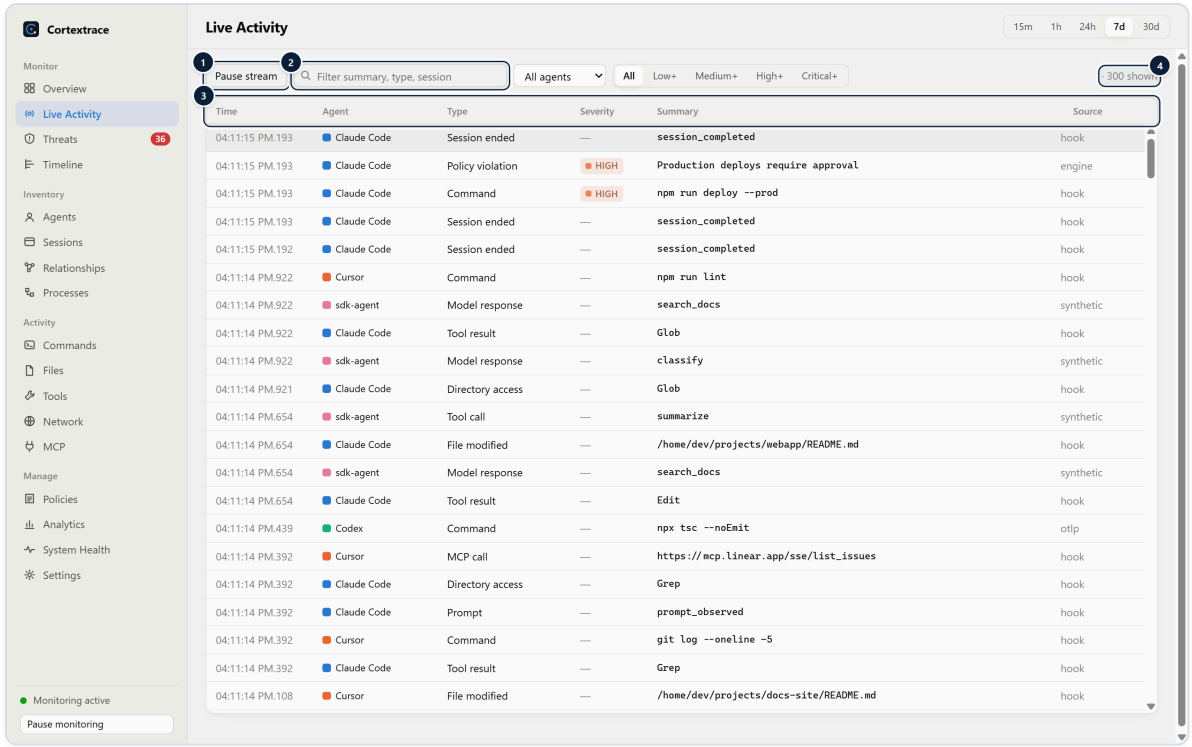
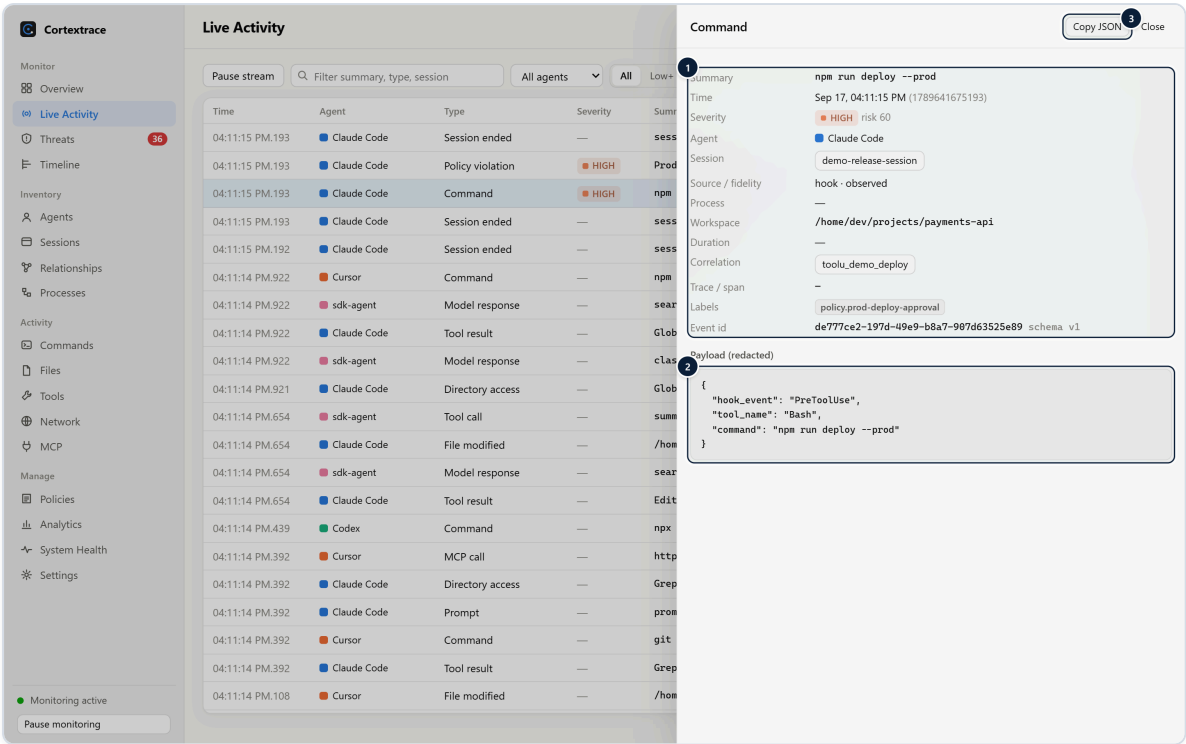


Figure 9 Live Activity: a virtualised stream of normalised events across all agents.

- 1 **Pause / resume** — Freezes the stream for inspection without stopping collection.
- 2 **Filter** — Client-side filters by text, agent and minimum severity.
- 3 **Event stream** — Virtualised list of normalised events: time (ms), agent, type, severity, summary and source (hook, otlp, process, synthetic...).
- 4 **Rate** — Observed events per second and the number of rows shown.



**Figure 10** Every event can be inspected with its schema fields and redacted payload.

- 1

**Normalised fields** — Schema fields: agent, session, source/fidelity, process, workspace, correlation and trace ids, labels.
- 2

**Redacted payload** — Type-specific payload after secret redaction and privacy minimisation.
- 3

**Copy JSON** — Copies the complete event for use in tickets or other tools.

## Commands and processes

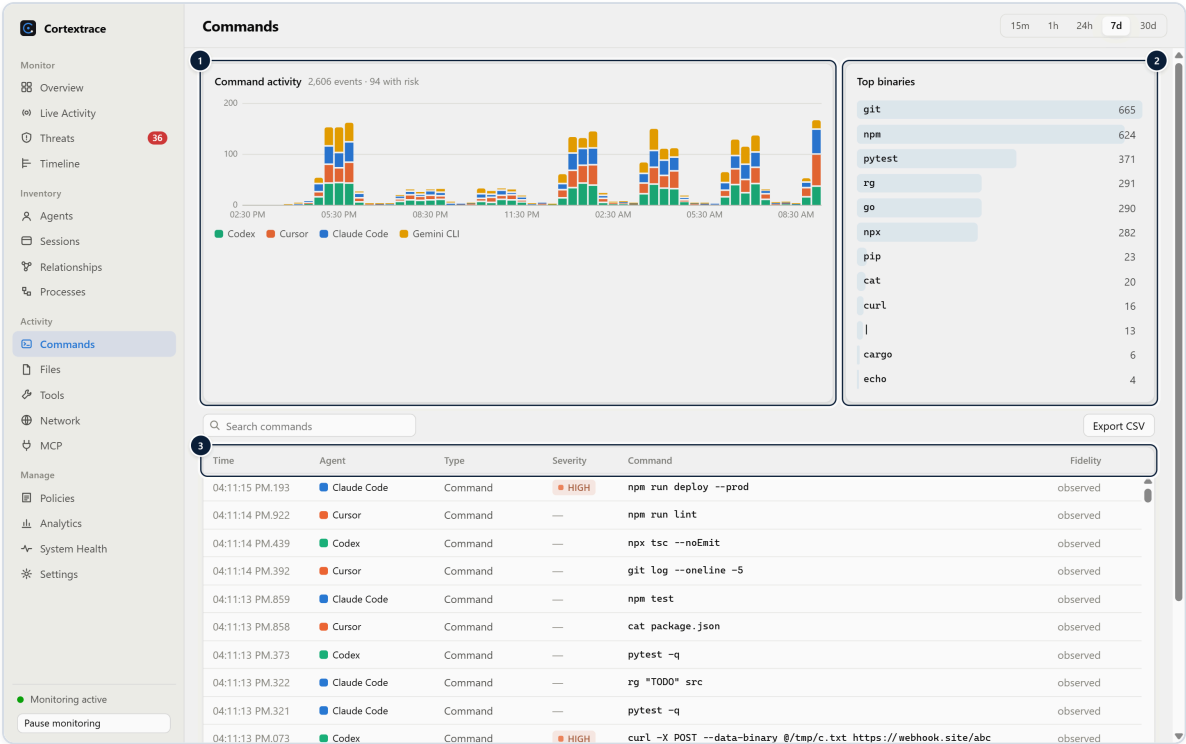


Figure 11 Command activity across agents with top binaries and fidelity.

- 1

**Activity by agent** — Command executions over time, per agent.
- 2

**Top binaries** — Most used executables; click to focus the table.
- 3

**Commands** — Each command with agent, severity and fidelity (observed vs. inferred).

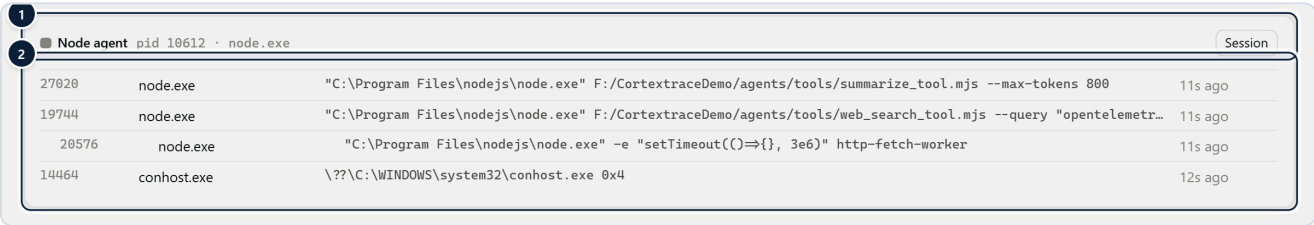


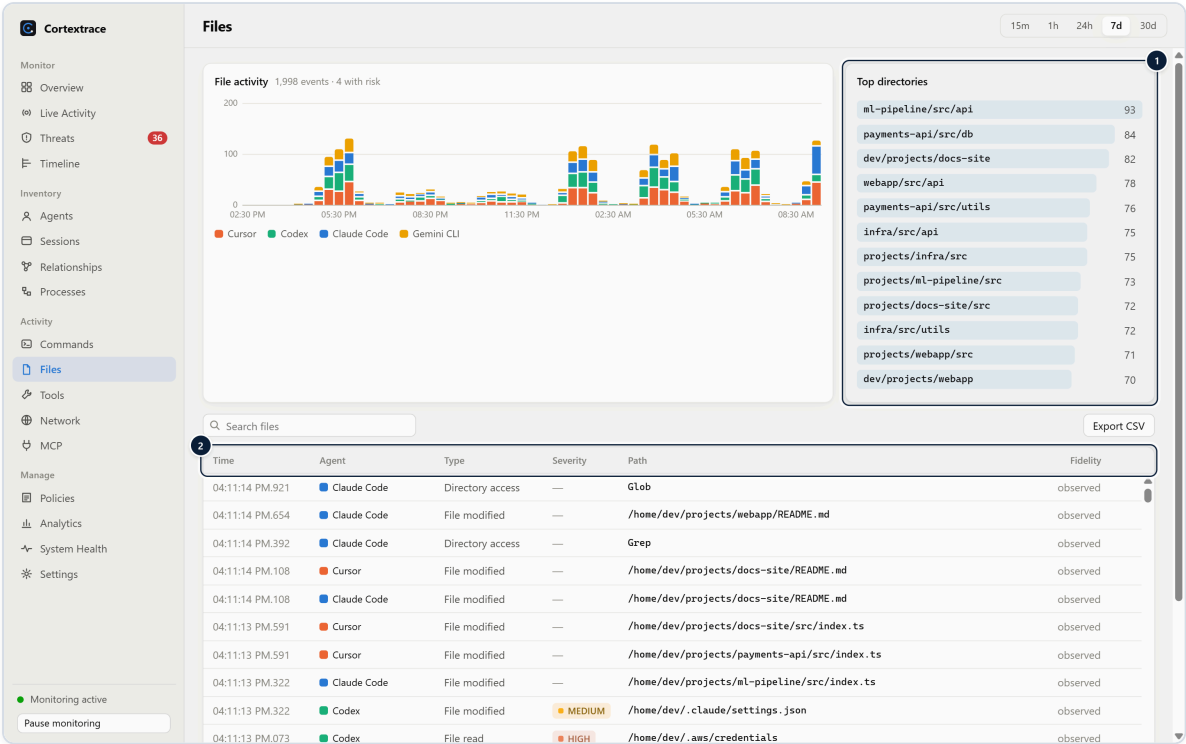
Figure 12 Process tree of a discovered agent (demonstration agent process), showing descendants with command lines.

- 1

**Agent root process** — A running agent identified by its process signature (here a Node.js agent framework), with pid and session link.
- 2

**Descendant tree** — Child and grandchild processes spawned by the agent, with pid, name, command line and age.

Files, network and MCP



**Figure 13** File operations: reads, modifications, creations, deletions and directory access. File contents are never stored.

- 1 Top directories** — Directories with the most read/write activity.
- 2 File operations** — Reads, modifications, creations, deletions and directory access with paths; contents are never stored.

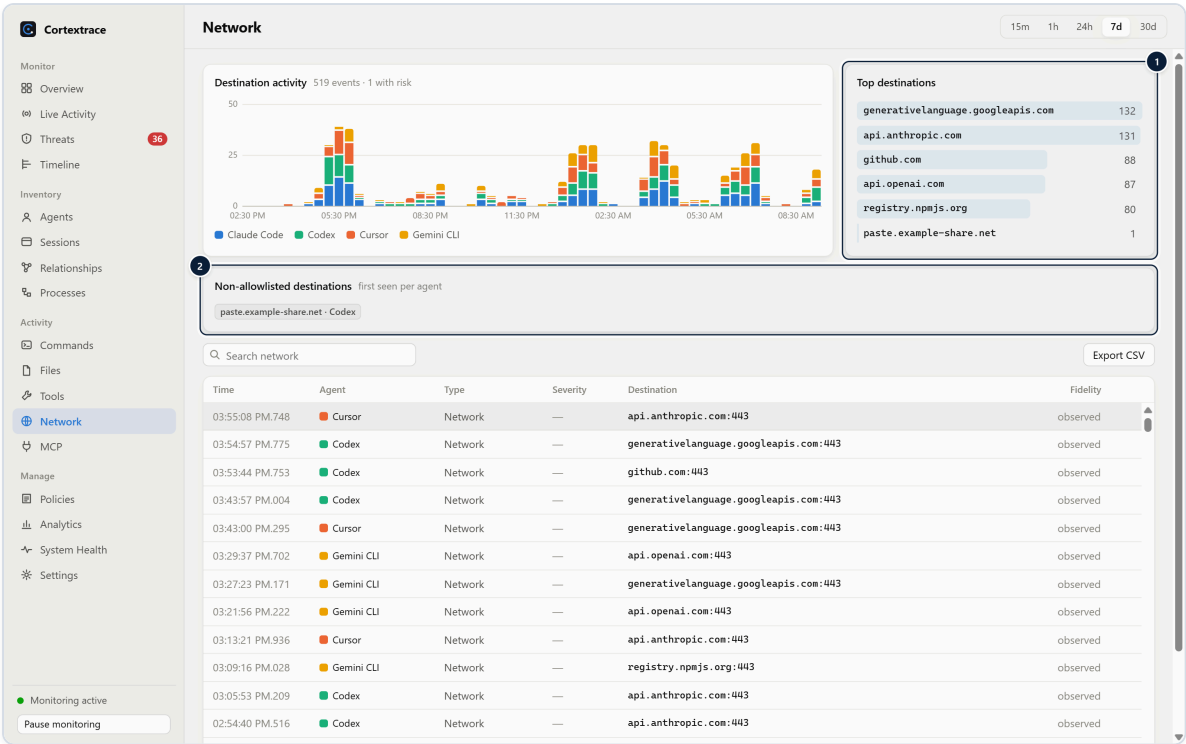


Figure 14 Network metadata: destinations contacted by agents and destinations outside the allowlist.

- 1 **Destinations** — Most contacted destinations for the range (connection metadata only: host/IP and port).
- 2 **First-seen destinations** — Destinations outside the allowlist, first seen per agent.

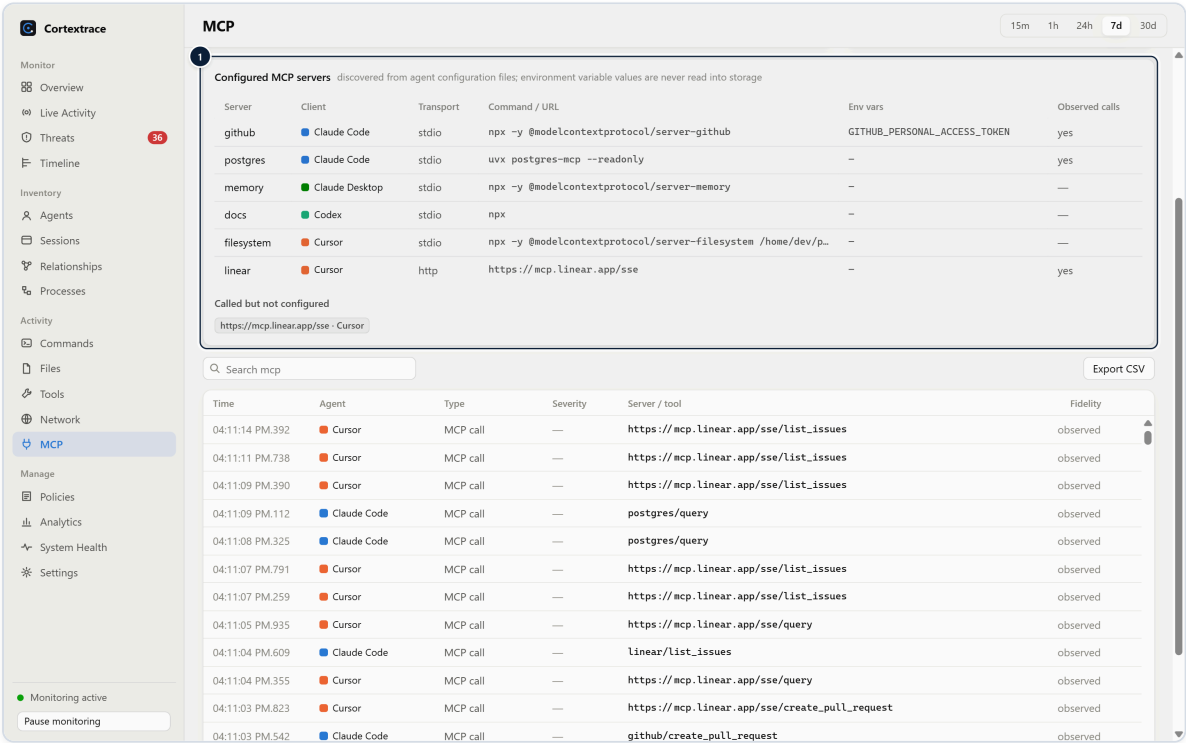


Figure 15 MCP inventory from agent configuration files; environment variable values are never stored.

- 1 **MCP inventory** — Servers found in agent configuration files: client, transport, command/URL and environment variable names (never values).

**Limitations**

Polling can miss processes that start and exit between snapshots; hook integrations capture those actions. File-change attribution is inferred from workspace ownership and marked as such. Network monitoring records connection metadata only — it does not inspect traffic.

## 09 Session Tracing

---

### Session timeline and replay

#### WHAT IT IS

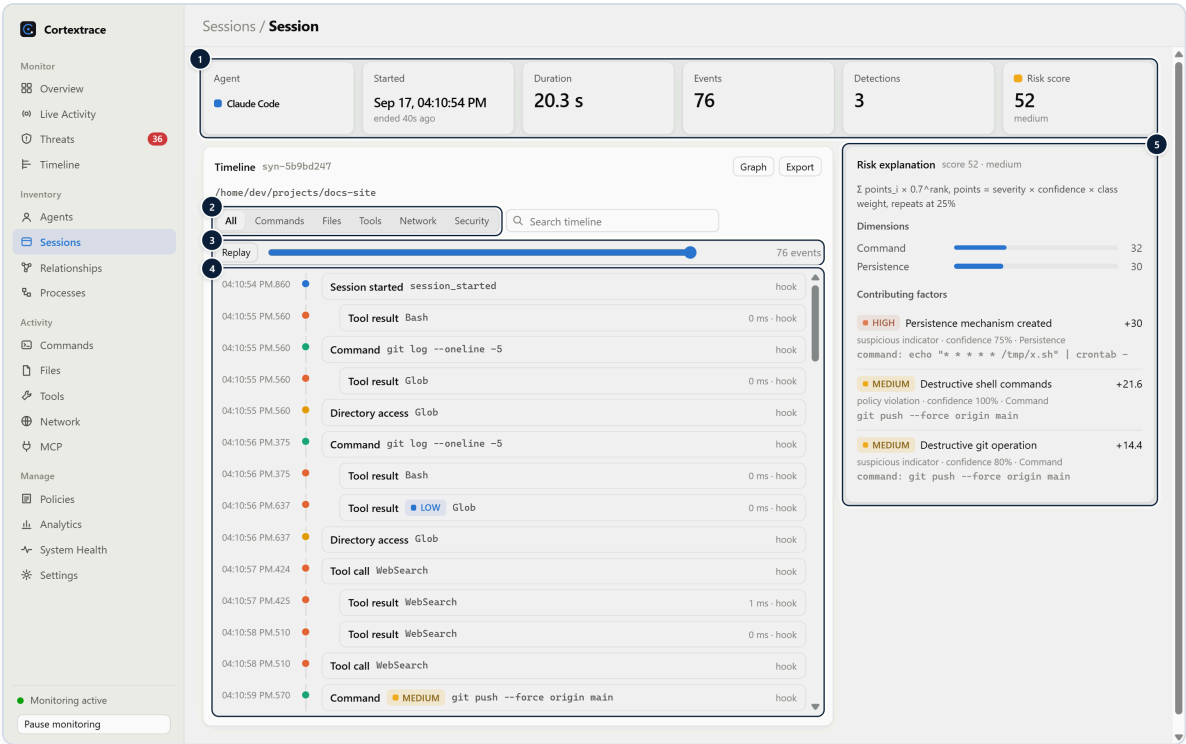
A chronological, filterable reconstruction of everything observed in one agent session, with a risk explanation alongside.

#### WHY IT MATTERS

Incidents are understood through sequence: what the agent read, ran and changed before and after a risky action.

#### HOW IT WORKS

- Sessions start with `session_started` (from hooks, OTLP or process discovery) and end with `session_completed` or process exit.
- The timeline shows millisecond timestamps, event type, severity, summary, duration and source; tool results and detections are indented under the events they correlate to.
- Event groups (commands, files, tools, network, security) and text search narrow the view; the replay control steps through events in order.
- Each timeline entry expands to its redacted payload and links to the raw event and its correlated parent.



**Figure 16** Session detail: summary, timeline with replay, and the risk explanation for the session.

- 1

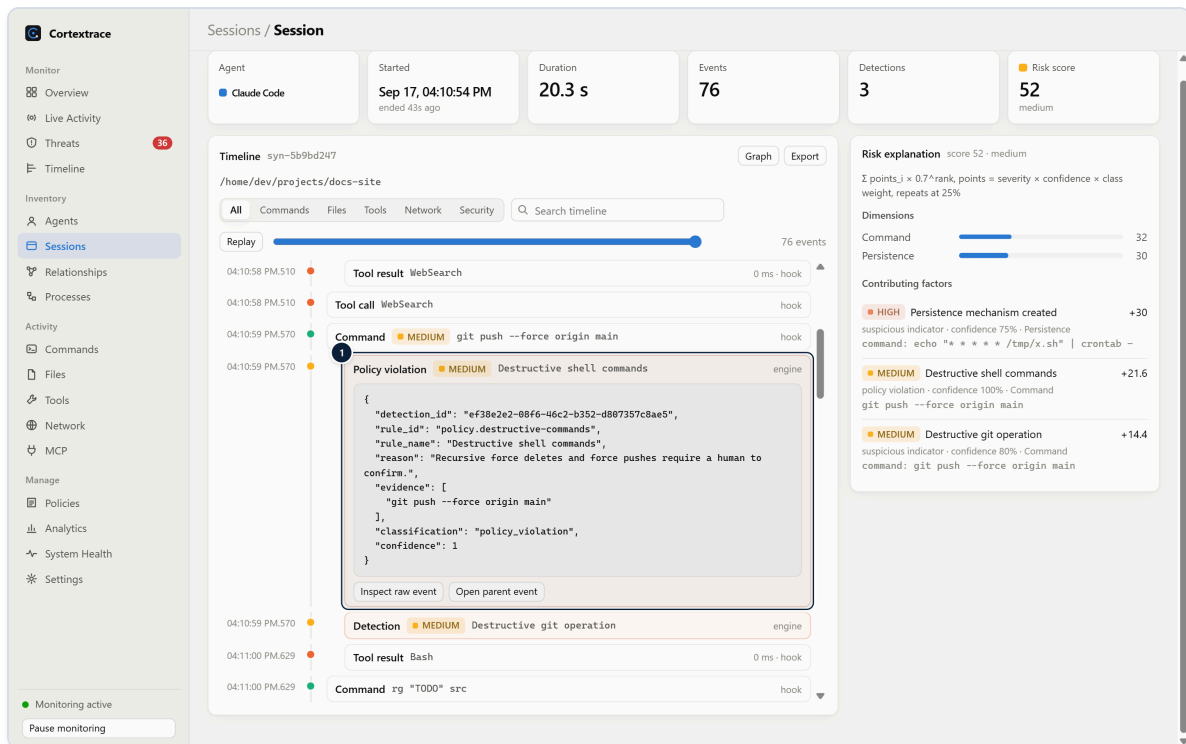
**Session summary** — Agent, start, duration, event count, detections and risk score.
- 2

**Event groups** — Filter the timeline to commands, files, tools, network or security events.
- 3

**Replay** — Step through the session in order; later events fade until reached.
- 4

**Timeline** — Millisecond timestamps, event type, severity, summary, duration and source; tool results and detections indent under their correlated call.
- 5

**Risk explanation** — Score, per-dimension breakdown and every contributing factor with points, confidence and evidence.



**Figure 17** An expanded detection event within the session timeline.

- 1 **Expanded detection** — Expanding an event shows its payload with links to the raw event and its parent (correlated) event.

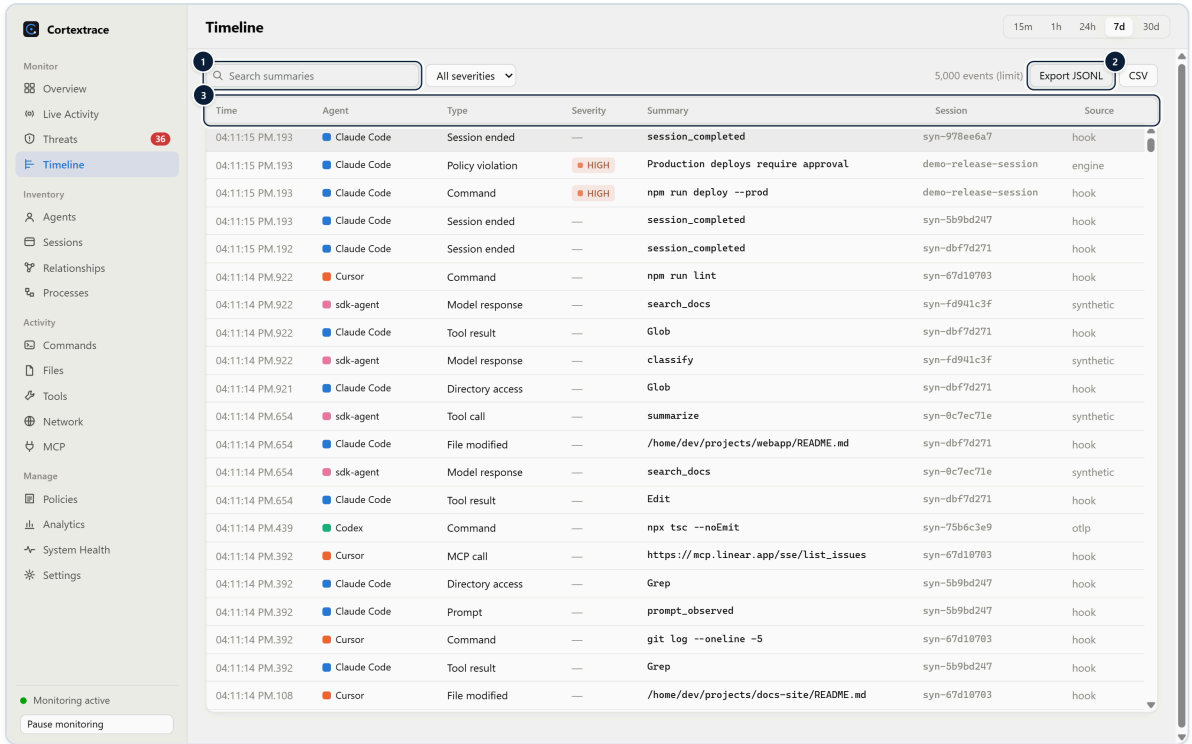


Figure 18 Global timeline across agents with search, severity filter and export to JSONL or CSV.

- 1

**Search & severity** — Server-side search across event summaries, filtered by minimum severity.
- 2

**Export** — Exports the current query as JSONL or CSV (CSV cells are neutralised against formula injection).
- 3

**Correlated events** — All agents in one timeline with session ids; drill-down filters (agent, session, correlation id, type, workspace) arrive from other pages.

# 10 Security Intelligence

Detection runs on every event as it is processed. Findings distinguish facts from indicators and from confirmed policy violations — nothing is labelled malicious from a pattern alone.



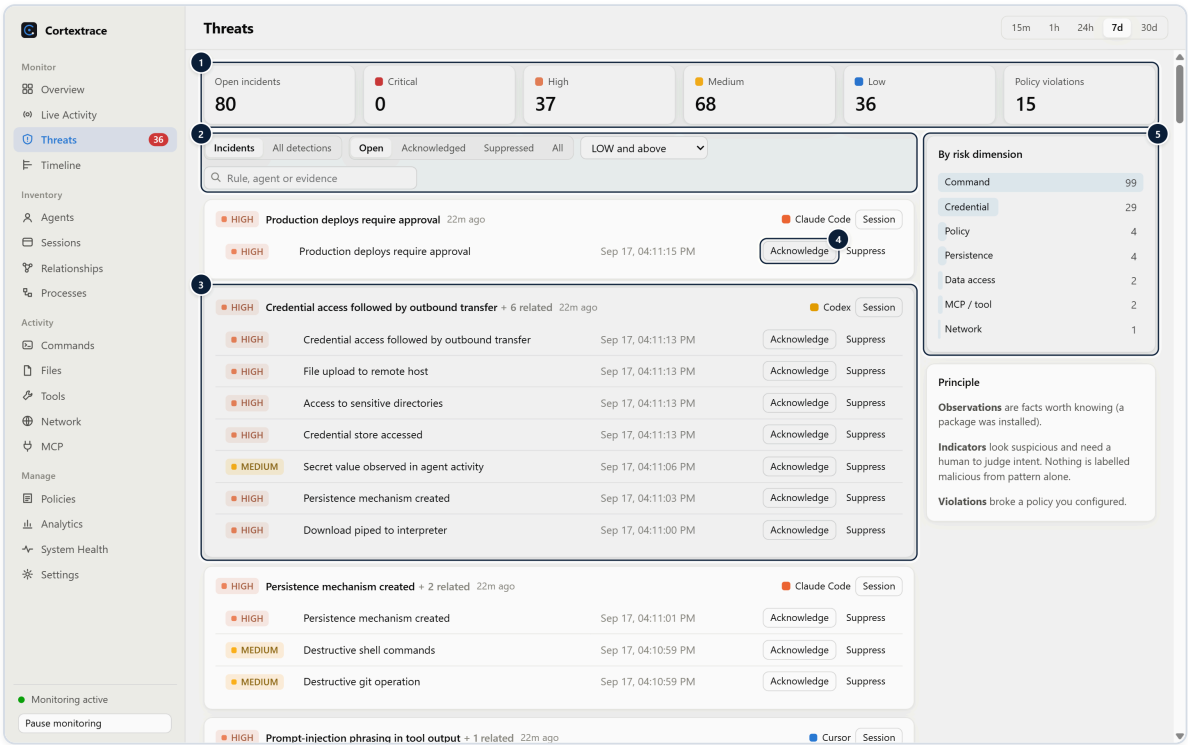
**Figure 19** Security detection pipeline: four detection layers, the detection record, and downstream timeline events, risk and alerts.

## Detection layers

| Layer                    | What it detects                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Output classification                                                     |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Deterministic rules (39) | Destructive commands, privilege elevation, download-and-execute, encoded or obfuscated commands, reverse-shell indicators, living-off-the-land binaries, persistence, disabled security controls, package installs, credential and secret access, cloud metadata access, uploads, sensitive files, agent control-file edits, CI/IaC edits, exfiltration services, prompt-injection phrasing, hidden Unicode, markdown-image exfiltration, sensitive tool arguments | Suspicious indicator or observation (per rule)                            |
| Secret exposure          | Secret formats discovered by redaction in tool input or output (values are never stored)                                                                                                                                                                                                                                                                                                                                                                           | Suspicious indicator                                                      |
| Policies                 | User-defined conditions on any event field                                                                                                                                                                                                                                                                                                                                                                                                                         | Observation (observe mode) or policy violation (alert / require approval) |
| Heuristics               | Mass file modification or deletion, large repository change, credential access followed by outbound transfer within 120 s, denied action executed                                                                                                                                                                                                                                                                                                                  | Suspicious indicator, observation or policy violation                     |

| Layer                  | What it detects                                                                                                                                     | Output classification               |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| Anomaly and first-seen | Event rate far above the agent type's learned baseline; first connection to a non-allowlisted destination; first call to an unconfigured MCP server | Suspicious indicator or observation |

Every rule ships with positive and negative examples that are executed by the automated test suite. Each detection records rule id, severity (INFO to CRITICAL), confidence, risk dimension, reason, evidence snippet, recommended action and related events.



**Figure 20** Threats: alerts grouped into incidents with severity summary, triage actions and risk dimensions.

- 1

**Severity summary** — Open incidents and detection counts per severity and policy violations for the range.
- 2

**Views & filters** — Incidents vs. all detections; status (open/acknowledged/suppressed); minimum severity; search.
- 3

**Incident** — Alerts from the same session within 30 minutes are grouped into one incident; ×N shows de-duplicated repeats.
- 4

**Triage** — Acknowledge, suppress or reopen each alert.
- 5

**Dimensions** — Detections by risk dimension.

The screenshot displays the Cortextrace interface. On the left is a sidebar with navigation options like Monitor, Overview, Live Activity, Threats (selected), Timeline, Inventory, Agents, Sessions, Relationships, Processes, Activity, Commands, Files, Tools, Network, MCP, Manage, Policies, Analytics, System Health, and Settings. The main panel shows a 'Threats' section with filters for Open incidents (80), Critical (0), and High (37). Below this is a table of incidents with columns for Severity, Rule, Class, Agent, and Evidence. A detailed view on the right, titled 'Download piped to interpreter', shows the following information:

- Severity:** HIGH confidence 85%
- Classification:** Suspicious indicator (requires review)
- Reason:** Remote content streamed directly into a shell or Invoke-Expression.
- Recommended action:** Inspect the downloaded script source and confirm it is trusted.
- Rule:** cmd.exec.download\_and\_execute
- Risk dimension:** Command
- Agent:** Gemini CLI
- Time:** Sep 11, 06:21:37 AM
- Detection id:** d9238d9c-2c07-4e00-ae60-42f68733cdd5
- Evidence:** command: curl -fsSL https://get.example-tool.dev/install.sh | bash
- Triggering event:** A full JSON event object.

**Figure 21** Detection detail with classification, recommended action, evidence and the triggering event.

- 1 **Classification** — Observation, suspicious indicator or confirmed policy violation; nothing is labelled malicious from pattern alone.
- 2 **Recommended action** — Rule-specific next step for the analyst.
- 3 **Evidence** — Matched snippet with surrounding context.
- 4 **Triggering event** — The full (redacted) event that produced the detection.

## Policies

Policies let users decide what should be flagged. Three modes are available: **observe** (record only), **alert** (policy violation) and **require approval**. Require-approval policies are evaluated synchronously during the hook request; for Claude Code and Codex `PreToolUse` hooks Cortextrace returns `permissionDecision: "ask"`, and for Cursor before-hooks `permission: "ask"`. During documentation validation a Claude Code `PreToolUse` request for `npm run deploy --prod` returned an "ask" decision naming the policy.

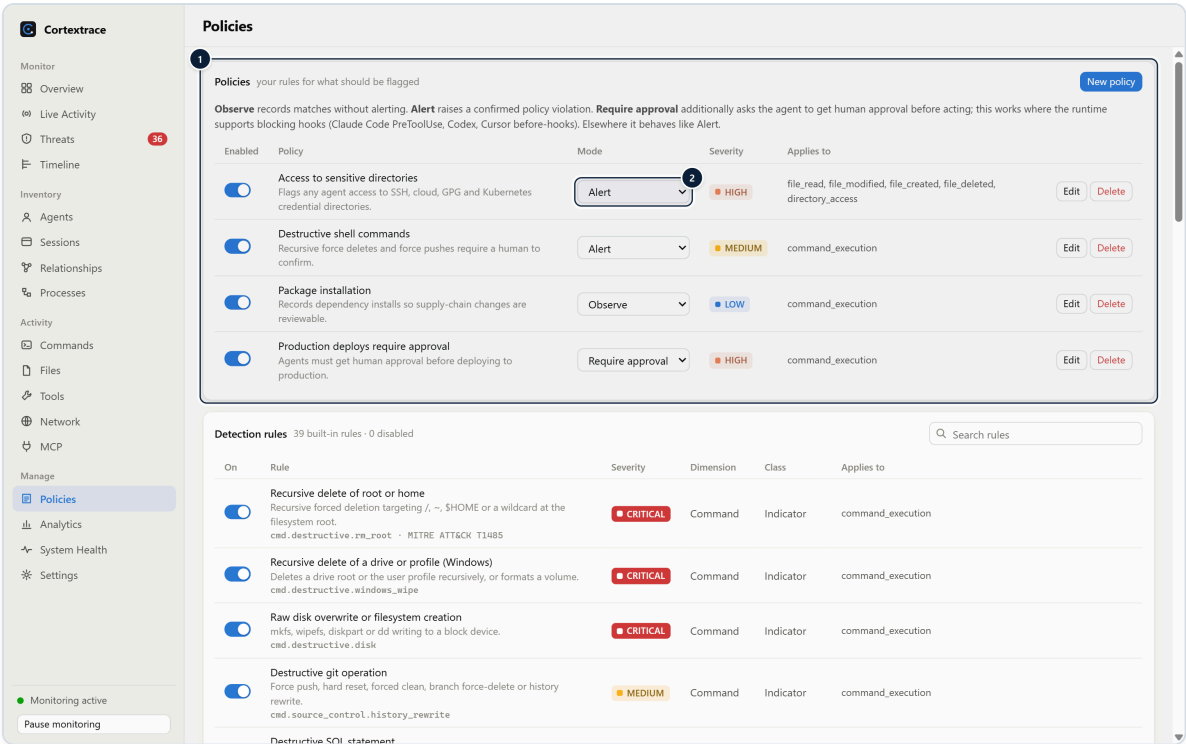


Figure 22 Policies with enable switch, mode, severity and scope.

- 1

**Policies** — User-defined policies with enable switch, mode (observe / alert / require approval), severity and scope; edited as validated JSON.
- 2

**Mode** — Require approval returns an "ask" decision to agents that support blocking hooks.

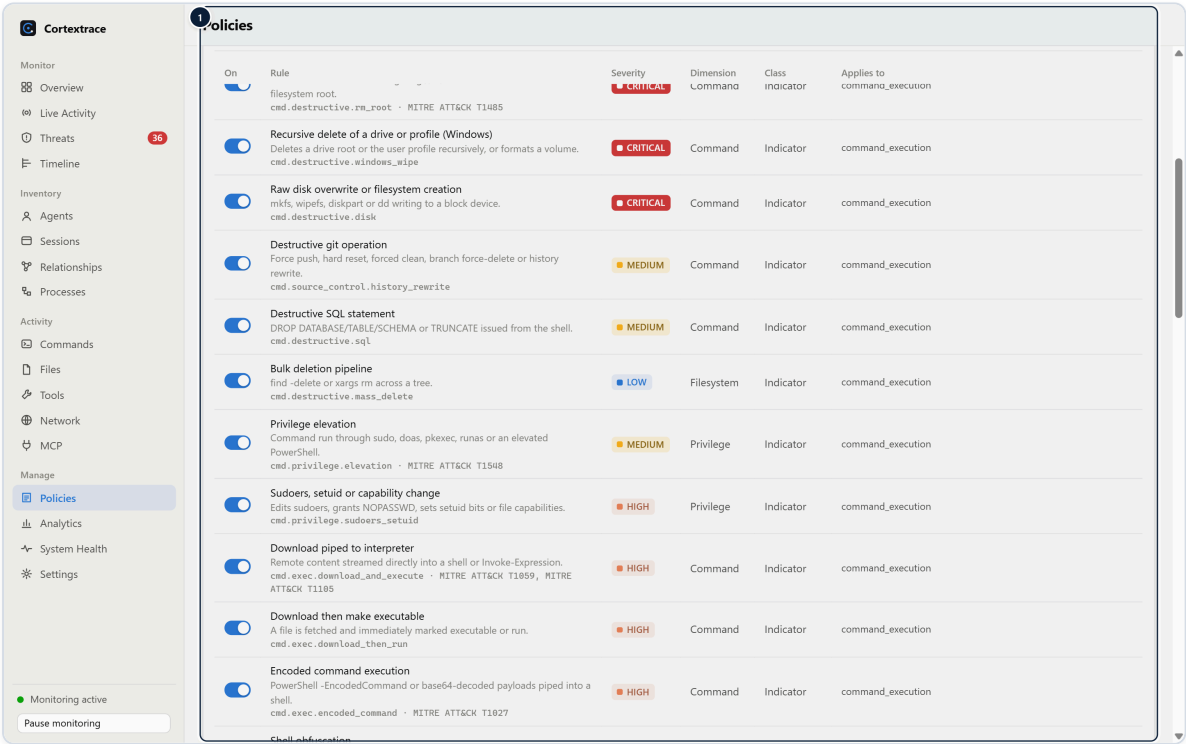


Figure 23 Built-in rule catalogue with per-rule enable switches.

- 1

**Rule catalogue** — Every built-in rule with description, severity, risk dimension, classification, references and a per-rule enable switch.

## Alerts and incidents

- Alerts are created for detections at or above a configurable severity (default MEDIUM).
- Repeated alerts with the same rule, session and normalised evidence within the de-duplication window increase a counter instead of notifying again.
- Alerts in the same session within 30 minutes share an incident.
- Alerts can be acknowledged, suppressed or reopened; suppressions can be scoped to an agent type.
- Channels: dashboard, desktop notifications (default HIGH and above), `alerts.jsonl` in the data directory, and an optional webhook that carries alert metadata only.

# 11 Agent Risk Analysis

## Explainable risk scoring

### WHAT IT IS

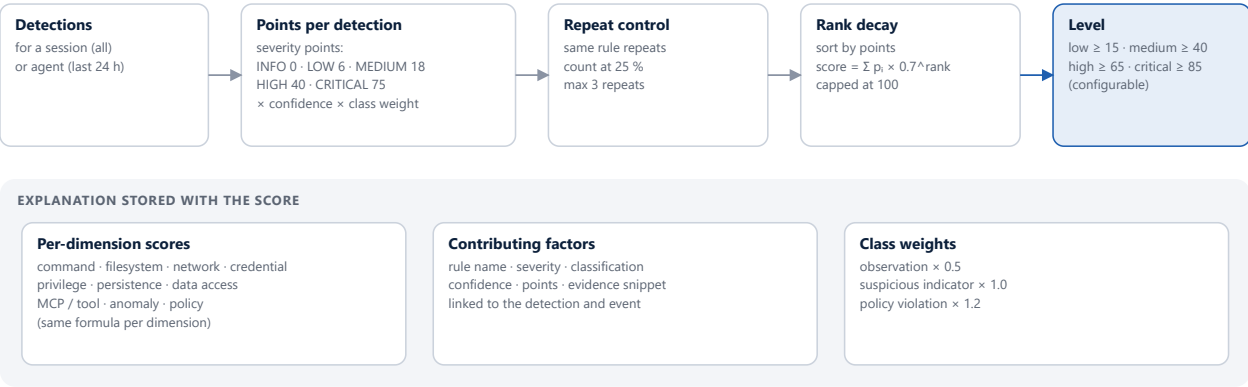
A 0–100 risk score for every session (all detections) and every agent (last 24 hours), with a per-dimension breakdown and the list of contributing factors.

### WHY IT MATTERS

A number without reasons cannot be acted on or trusted. Every point in a Cortextrace score traces back to a specific detection and its evidence.

### HOW IT WORKS

- Each detection contributes **severity points × confidence × classification weight**. Severity points: INFO 0, LOW 6, MEDIUM 18, HIGH 40, CRITICAL 75. Weights: observation 0.5, suspicious indicator 1.0, policy violation 1.2.
- Repeats of the same rule count at 25%, at most three times.
- Contributions are sorted and combined as  $\sum p_i \times 0.7^{\text{rank}}$ , capped at 100, so a single critical finding dominates while many small findings still accumulate.
- The same formula yields scores for 10 dimensions: command, filesystem, network, credential, privilege, persistence, data access, MCP/tool, anomaly and policy.
- Levels are configurable; defaults: low ≥ 15, medium ≥ 40, high ≥ 65, critical ≥ 85.



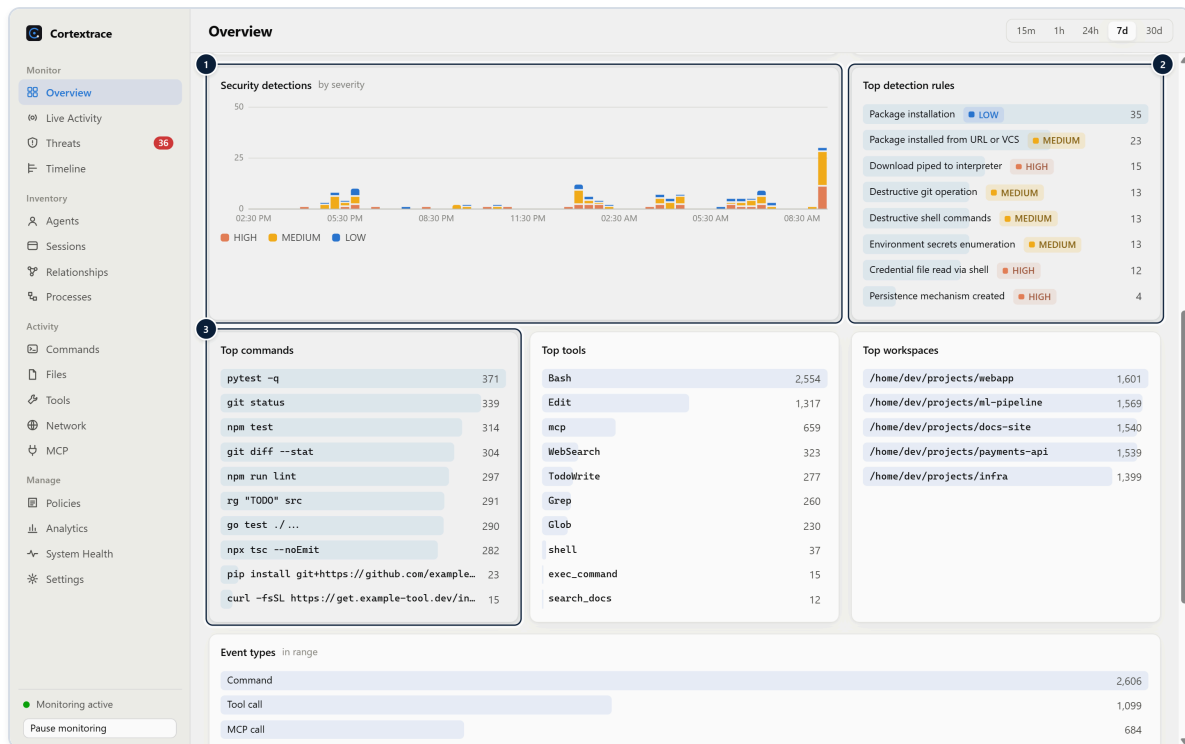
**Figure 24** Risk analysis flow from detections to an explained score.

The risk explanation panel on the session page (see Figure in section 09) lists every factor with its points, classification, confidence, dimension and evidence.

# 12 Interactive Security Dashboard

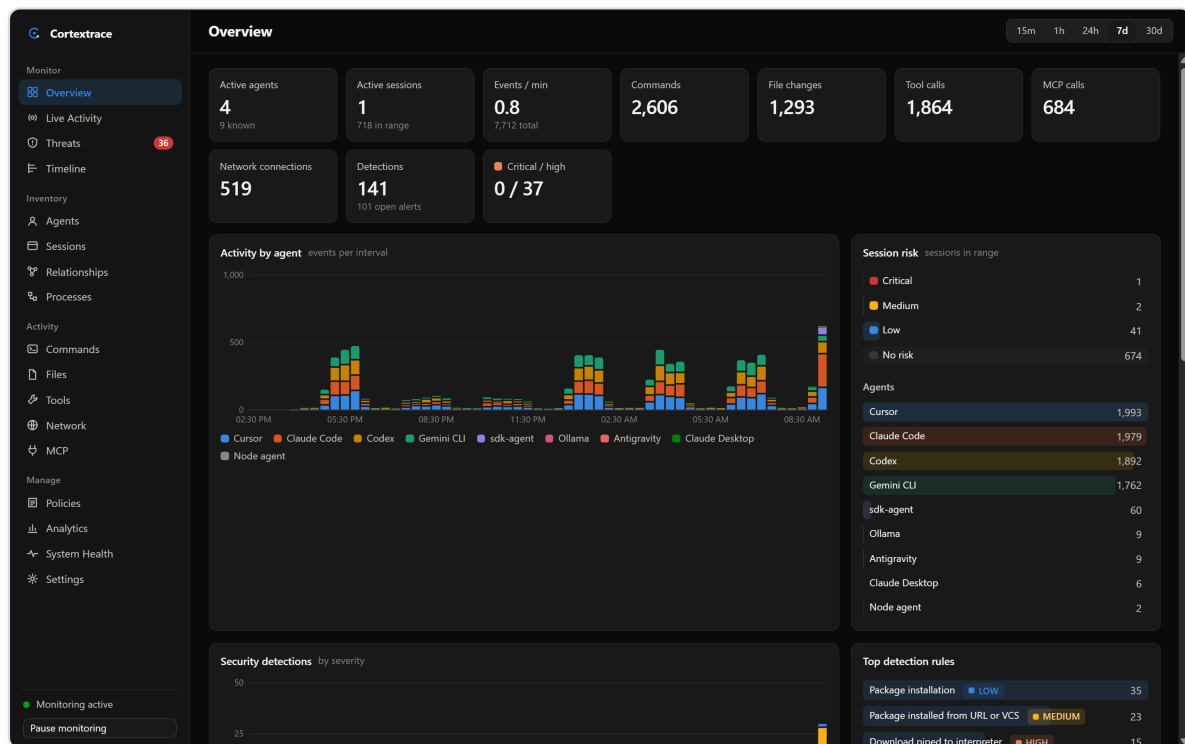
The dashboard is organised into Monitor (Overview, Live Activity, Threats, Timeline), Inventory (Agents, Sessions, Relationships, Processes), Activity (Commands, Files, Tools, Network, MCP) and Manage (Policies, Analytics, System Health, Settings).

| Capability               | Implementation                                                                                                                                                                       |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| KPIs                     | Active agents and sessions, events per minute, commands, file changes, tool and MCP calls, network connections, detections, critical/high counts; each tile links to its detail page |
| Charts                   | Stacked time-series columns with per-interval hover tooltips and toggleable legends; bar lists; hour-of-week heatmap; duration histogram                                             |
| Time range               | Global selector: 15 minutes, 1 hour, 24 hours, 7 days, 30 days                                                                                                                       |
| Filters and search       | Severity, agent, event group, classification, status and text search; server-side search on the Timeline                                                                             |
| Drill-down               | Chart columns open the Timeline for that interval; top lists open filtered views; events link to sessions and correlated events                                                      |
| Live updates             | Coalesced event stream ( $\leq 1,000$ events per 500 ms) while the window is visible; pause and resume on Live Activity; pages refresh periodically                                  |
| Export                   | Events as JSONL or CSV; diagnostics bundle without events or tokens                                                                                                                  |
| Themes and accessibility | Light and dark themes (system, light or dark), reduced-motion and reduced-transparency support, keyboard-focusable controls                                                          |



**Figure 25** Overview (lower section): detections by severity, top rules and top commands.

- 1 Detections by severity** — Detections over time, stacked by severity.
- 2 Top rules** — Most frequent rules with severity; click to filter the Threats view.
- 3 Top commands** — Most frequent commands, tools and workspaces for the range.



**Figure 26** The same Overview in the dark theme.

# 13 Agent Relationship Graph

## Relationship graph

### WHAT IT IS

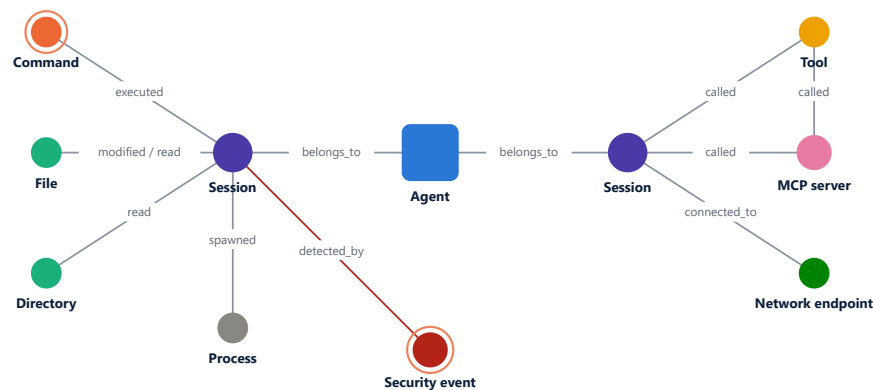
An interactive force-directed graph of agents, sessions, processes, commands, files, directories, tools, MCP servers, network endpoints and security events.

### WHY IT MATTERS

Relationships reveal what tabular lists hide: which session touched which files, which MCP server exposed which tools, and where detections cluster.

### HOW IT WORKS

- The graph is built from up to 3,000 events in the selected scope (time range, one agent or one session) and keeps the 400 most connected or risky nodes.
- Edges are typed: belongs\_to, spawned, executed, read, modified, called, connected\_to and detected\_by.
- Node size reflects occurrences; a ring marks nodes with elevated risk. Nodes can be dragged; the canvas pans and zooms; selecting a node highlights its neighbours.



Nodes: agent, session, process, command, file, directory, tool, MCP server, network endpoint, security event. Node size  $\propto$  occurrences; ring = elevated risk ( $\geq 40$ ).

**Figure 27** Relationship graph concept: node types and typed edges.

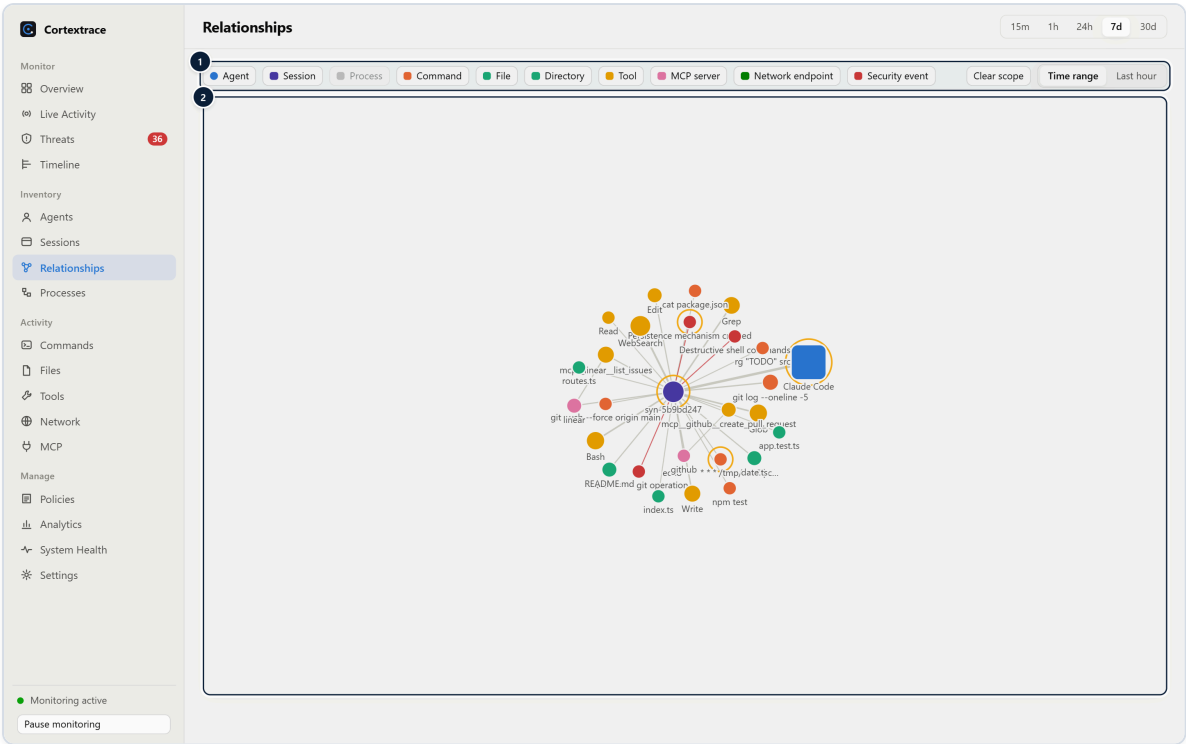


Figure 28 Relationship graph scoped to one session.

- 1 **Node types** — Toggle agents, sessions, processes, commands, files, directories, tools, MCP servers, network endpoints and security events.
- 2 **Force-directed graph** — Nodes are sized by occurrences; rings mark elevated risk; edges are typed (executed, read, modified, called, connected\_to, detected\_by). Drag nodes, pan and zoom.

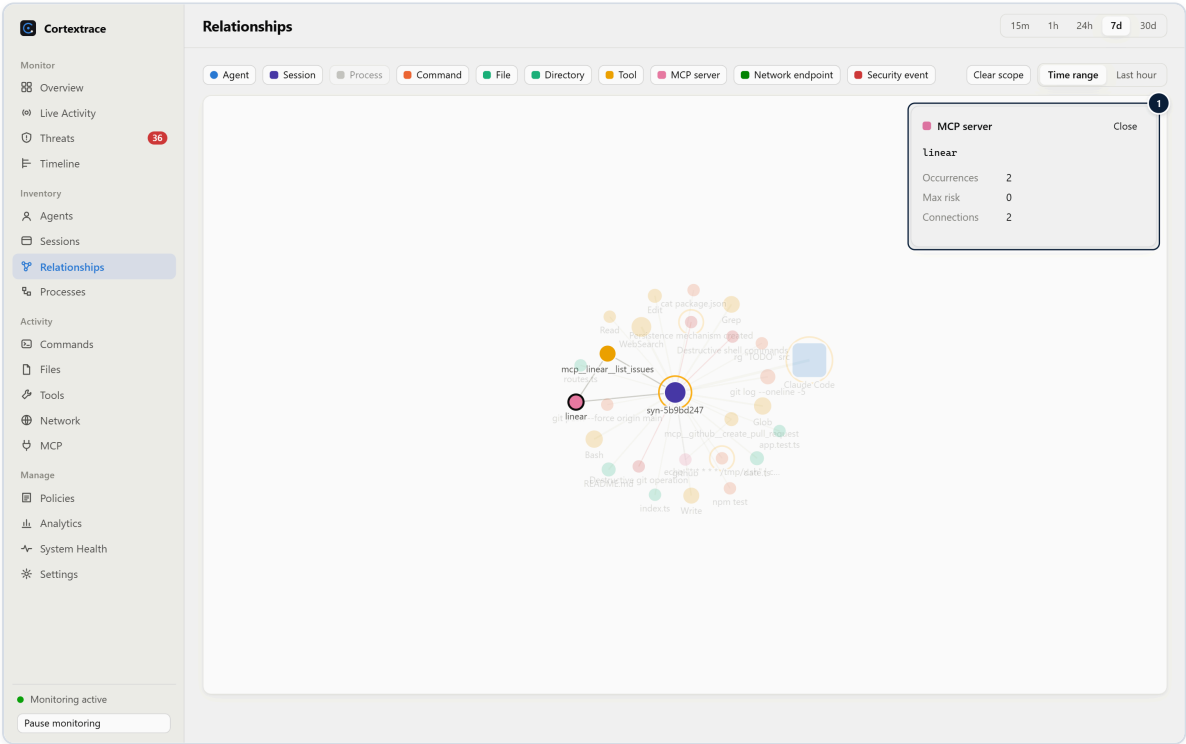


Figure 29 Selecting a node shows its details and highlights direct neighbours.

- 1 **Selection** — Selecting a node highlights its direct neighbours and shows occurrences, maximum risk and connection count, with navigation actions.

# 14 Analytics

Analytics summarise behaviour over time for the selected range, using the same local database.

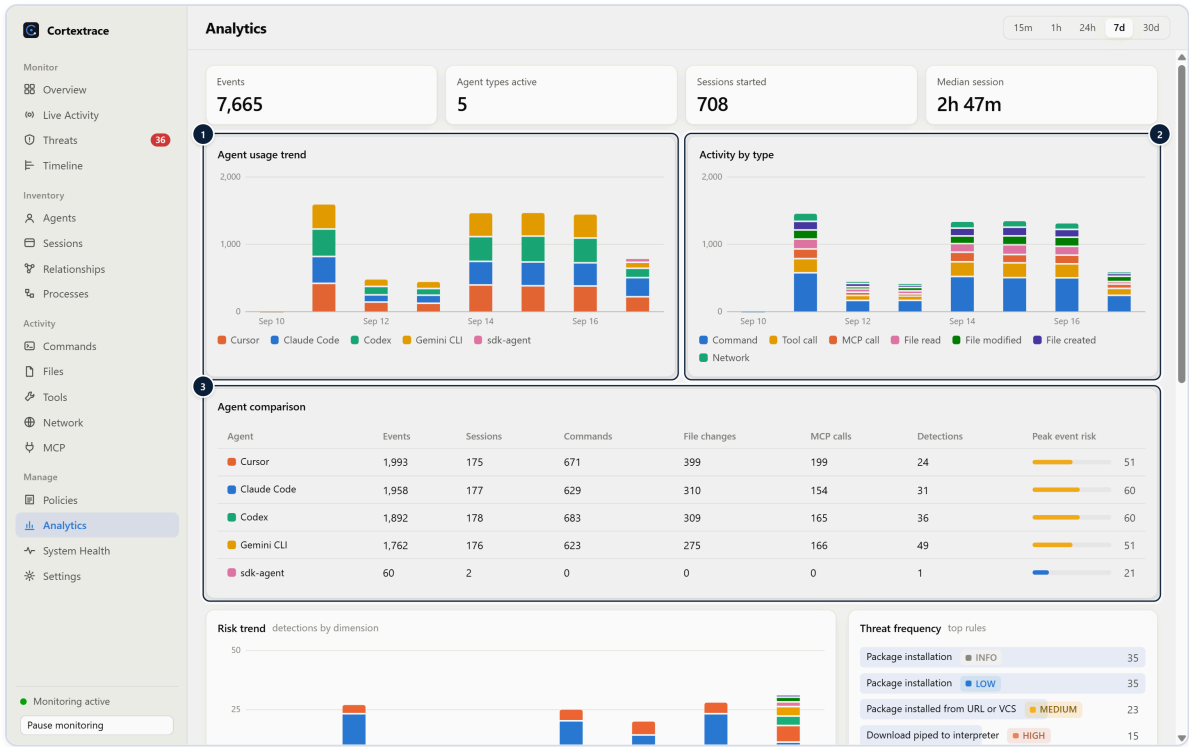


Figure 30 Agent usage trend, activity by type and per-agent comparison.

- 1

Usage trend — Events per agent per interval.
- 2

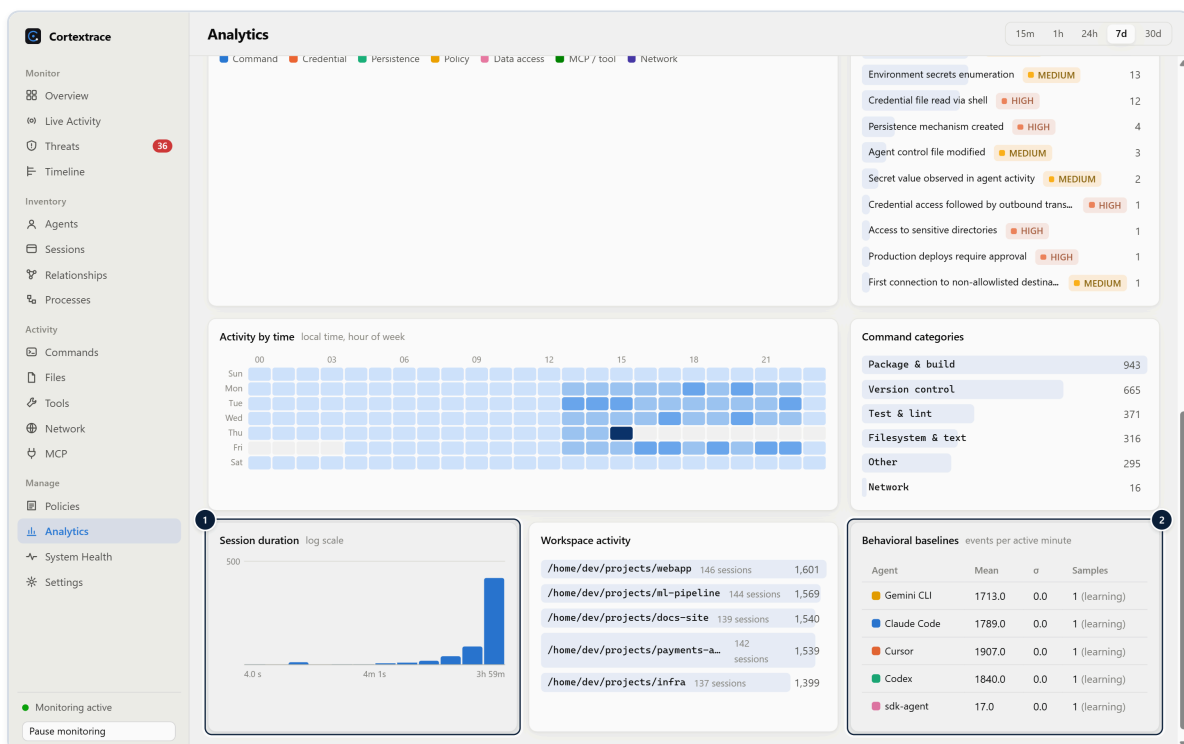
Activity by type — Commands, file operations, tool and MCP calls, network over time.
- 3

Agent comparison — Per-agent totals: events, sessions, commands, file changes, MCP calls, detections and peak event risk.



**Figure 31** Risk trend by dimension, threat frequency, hour-of-week activity and command categories.

- 1 Risk trend** — Detections over time by risk dimension.
- 2 Activity heatmap** — Hour-of-week activity in local time.
- 3 Command categories** — Commands grouped into categories such as version control, package & build, network.



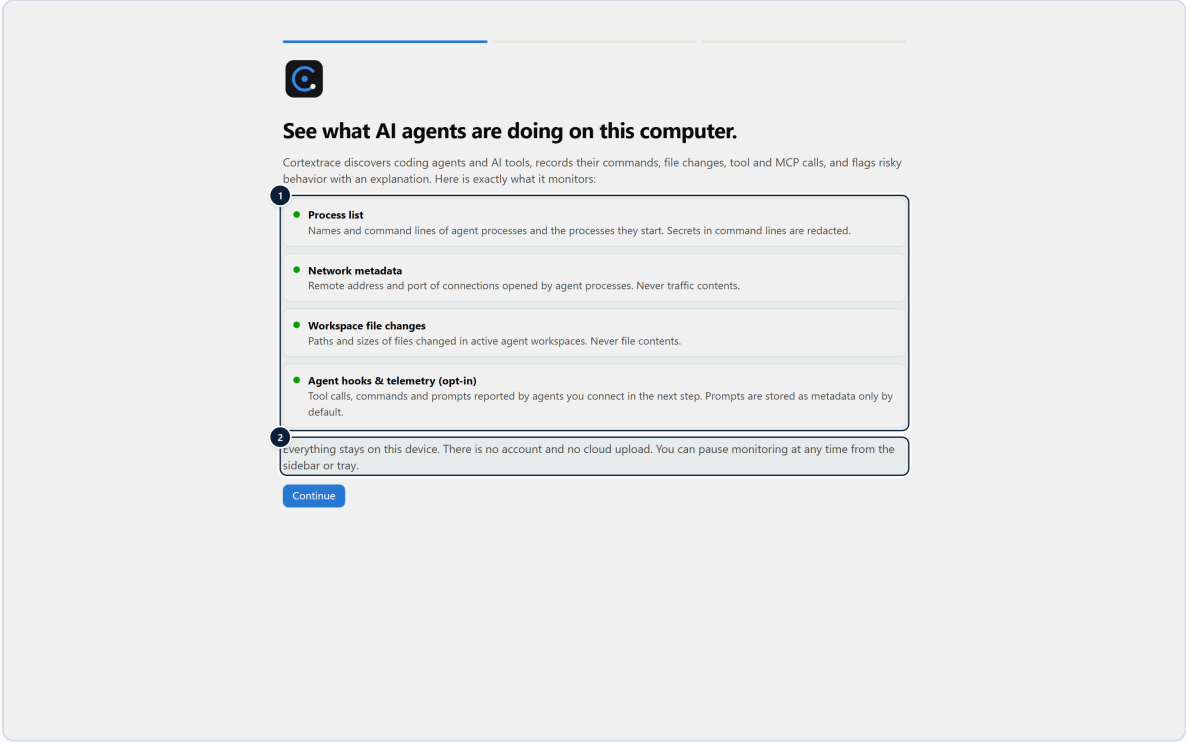
**Figure 32** Session duration distribution, workspace activity and learned behavioural baselines.

- 1 Session durations** — Distribution of session durations (log-scaled bins).
- 2 Baselines** — Learned events-per-minute baseline per agent type used by the anomaly detector (learning until 30 samples).

| Analysis                         | What it shows                                                                                                               |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Agent usage and activity by type | Events per interval per agent and per event type (commands, file operations, tool and MCP calls, network)                   |
| Agent comparison                 | Events, sessions, commands, file changes, MCP calls, detections and peak event risk per agent                               |
| Risk trend and threat frequency  | Detections over time by risk dimension; most frequent rules with severity                                                   |
| Activity by time                 | Hour-of-week heatmap in local time                                                                                          |
| Command categories               | Version control, package & build, test & lint, filesystem & text, network, cloud & containers, interpreters & shells, other |
| Session durations and workspaces | Distribution of session durations; most active workspaces with session counts                                               |
| Behavioural baselines            | Learned events-per-minute mean and deviation per agent type used by anomaly detection (learning until 30 samples)           |

# 15 Privacy and Security Architecture

Cortextrace is an observability and defensive security tool. It runs visibly, uses no elevated privileges, and keeps data on the device.



**Figure 33** The first-run screen discloses every data source before monitoring is used.

- 1

**Every data source disclosed** — Lists each monitored data source and what is never collected, before monitoring is used.
- 2

States the local-first model: no account and no cloud upload; pause is always available.

## Data collection boundaries

| Source                  | Collected                                                                                                                | Never collected                                            |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| Process list            | Name, executable path, command line (secrets redacted), pid/ppid, start time — for agent processes and their descendants | Unrelated processes are evaluated but not stored           |
| Connection tables       | Remote IP, port, resolved LLM API hostname for monitored agent processes                                                 | Traffic contents, DNS queries, other applications' traffic |
| Workspace file watching | Path, size, created / modified / deleted                                                                                 | File contents                                              |
| Agent hooks (opt-in)    | Tool name, command, file path, MCP server and tool, prompt length and hash                                               | File contents from read hooks (discarded on arrival)       |

| Source                  | Collected                                                                      | Never collected                                                   |
|-------------------------|--------------------------------------------------------------------------------|-------------------------------------------------------------------|
| OpenTelemetry (opt-in)  | Event names, tool names and durations, token counts, cost                      | Prompts — integrations are installed with prompt logging disabled |
| MCP configuration files | Server names, transport, command or URL (redacted), environment variable names | Environment variable values                                       |

## Redaction and minimisation

- Before storage, every payload string is scanned for private keys, AWS access keys, GitHub tokens, Anthropic and OpenAI keys, Google API keys, Slack tokens, Stripe live keys, JWTs, bearer tokens, credentials in URLs and `password=` / `token:` -style assignments; sensitive JSON keys such as `authorization` and `api_key` are masked entirely. Users can add custom patterns.
- Strings are truncated at 4 KB after redaction.
- Default retention modes: prompts as **metadata** (length and a 16-character SHA-256 prefix), tool output as **metadata** (length), command lines **redacted**. Detection runs before minimisation, so a prompt-injection phrase can be flagged without retaining the prompt.
- Events are retained for 30 days (maximum 5 million events) by default; both limits are configurable.

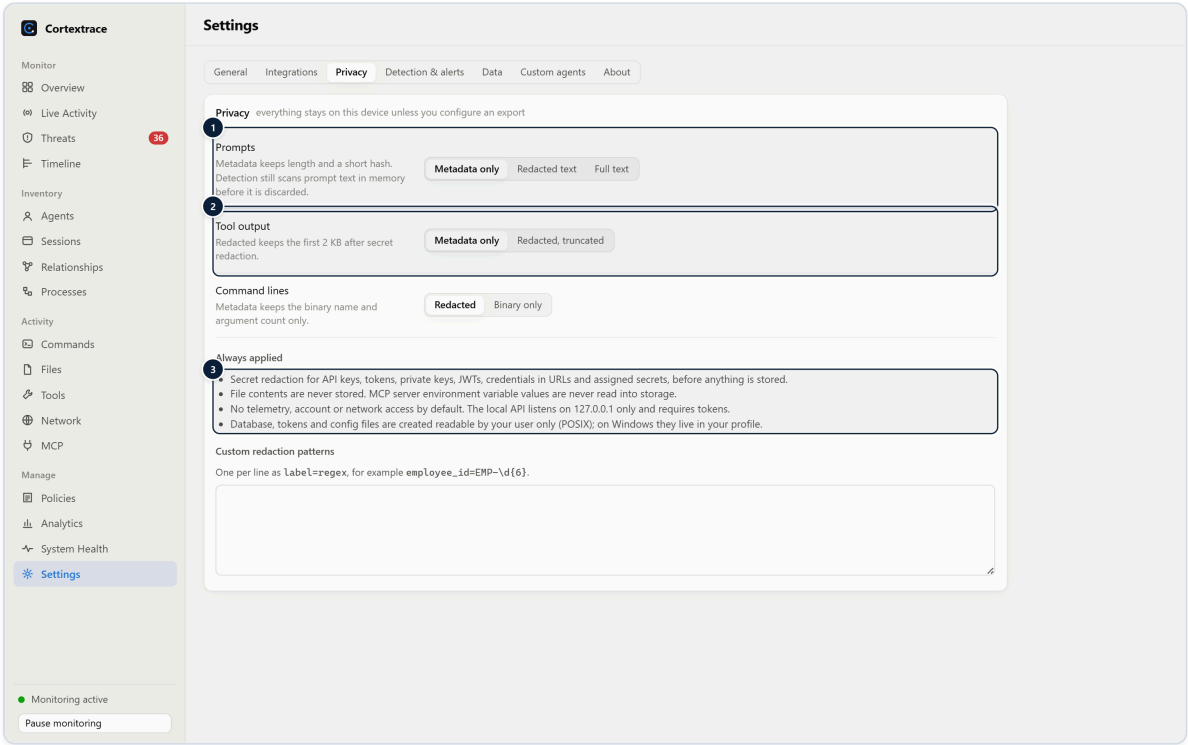


Figure 34 Privacy settings: retention modes and protections that are always applied.

- 1

**Prompt retention** — Metadata only (default), redacted text or full text.
- 2

**Tool output** — Metadata only (default) or redacted and truncated to 2 KB.
- 3

**Always applied** — Protections that cannot be switched off: secret redaction, no file contents, no outbound traffic by default, loopback-only API.

## Security controls

| Area             | Control                                                                                                                                                                                                                                                                         |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Local API        | Binds to 127.0.0.1; rejects requests with an Origin header or a non-loopback Host header (browser CSRF and DNS rebinding); JSON-only bodies; separate write-only ingest token and read/manage token compared in constant time; rate limiting; body size caps; schema validation |
| Desktop renderer | Context isolation, sandbox, Node integration disabled, allow-listed IPC channels with argument validation and sender checks, blocked navigation and new windows, all permission requests denied, strict Content Security Policy                                                 |
| Data at rest     | Database, configuration and tokens in the user profile; created with owner-only permissions on POSIX systems. The database is not encrypted; OS disk encryption is recommended                                                                                                  |
| Outbound traffic | None by default except DNS lookups used to recognise LLM API endpoints. Webhook and OTLP export are opt-in and visible in Settings                                                                                                                                              |
| Integrations     | Agent configuration is only changed on explicit user action, always after a backup; uninstall removes only Cortextrace's own entries                                                                                                                                            |
| User control     | Pause monitoring from the sidebar or tray (incoming data is discarded while paused); per-collector switches; retention and export controls                                                                                                                                      |

# 16 Technical Architecture

| Layer          | Technology                                                                                   |
|----------------|----------------------------------------------------------------------------------------------|
| Desktop shell  | Electron 44 (Node.js 24.21 runtime)                                                          |
| Language       | TypeScript; the engine has no Electron dependency and also runs headless under Node.js 24    |
| User interface | React 19 bundled with Vite; SVG charts; d3-force for the relationship graph                  |
| Storage        | SQLite via the built-in <code>node:sqlite</code> module (WAL mode) — no native add-ons       |
| Validation     | zod schemas for events, configuration and API payloads                                       |
| Packaging      | electron-builder: NSIS and MSI (Windows), DMG and PKG (macOS), AppImage, DEB and RPM (Linux) |

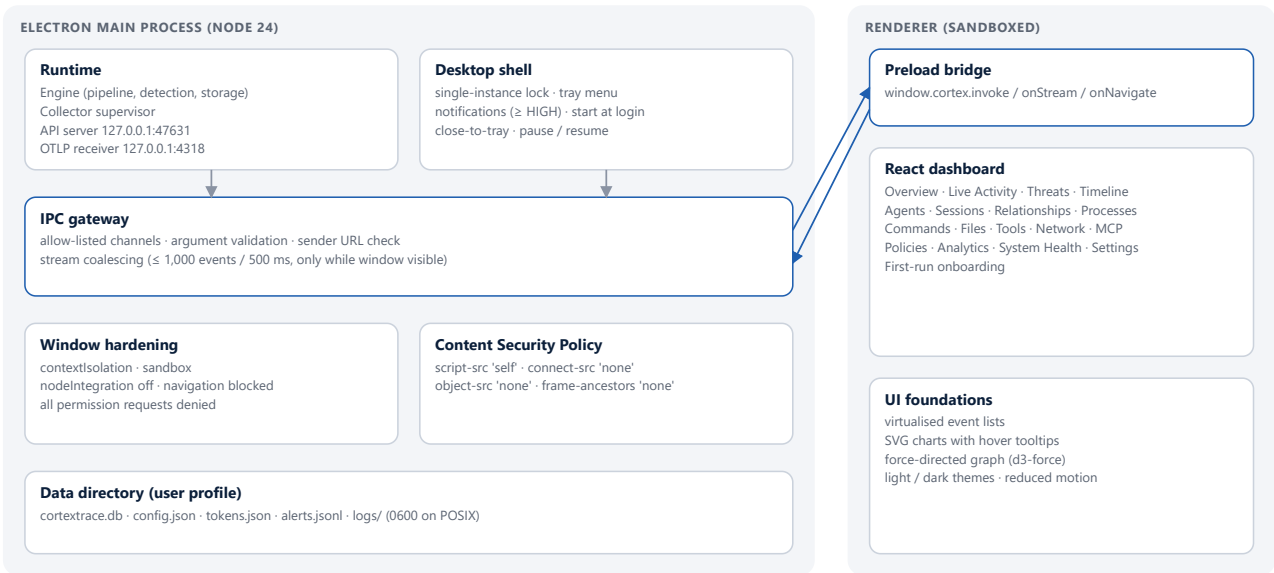


Figure 35 Desktop application architecture: main process, IPC gateway and sandboxed renderer.

## Storage

Events, sessions, agents, detections, alerts, inventory and baselines are stored in one SQLite database with indexes on time, session, agent, event type, agent type, workspace, tool and correlation id. Schema changes are versioned through migrations. Integrity is checked at startup and daily; if the database cannot be opened or fails the check, it is moved aside and a fresh database is created, and the dashboard shows a banner.

## Resilience

- Each collector runs on its own schedule; a failing collector is marked degraded and retried with exponential backoff without affecting others.
- An adapter that throws is counted in System Health; other adapters continue.

- The ingest queue is bounded at 50,000 events; overflow is dropped and counted rather than exhausting memory.
- A failed batch is logged and counted; the pipeline continues with the next batch.
- If the API or OTLP port is in use, local monitoring continues and the dashboard explains which endpoint is unavailable.

## Measured performance

Measured on Windows 11 with `npm run bench` (200,000 synthetic events across 150 sessions, with a detection rate far above typical real-world activity):

| Measurement                                  | Result                                  |
|----------------------------------------------|-----------------------------------------|
| Burst ingestion through the full pipeline    | ≈ 2,500 events/s (≈ 149,000 events/min) |
| Sustained 12,000 events/min for 20 s         | 0 events dropped                        |
| Latest 200 events                            | 6.6 ms                                  |
| Session timeline, 5,000 rows                 | 44 ms                                   |
| Type filter and text search, 234,000 rows    | 152 ms                                  |
| Relationship graph, 3,000 events             | 64 ms                                   |
| Overview aggregates, 150,000 events in range | 0.28 s                                  |
| Overview aggregates, 234,000 events in range | 2.1 s (results cached for 10 s)         |

Observability of the observer

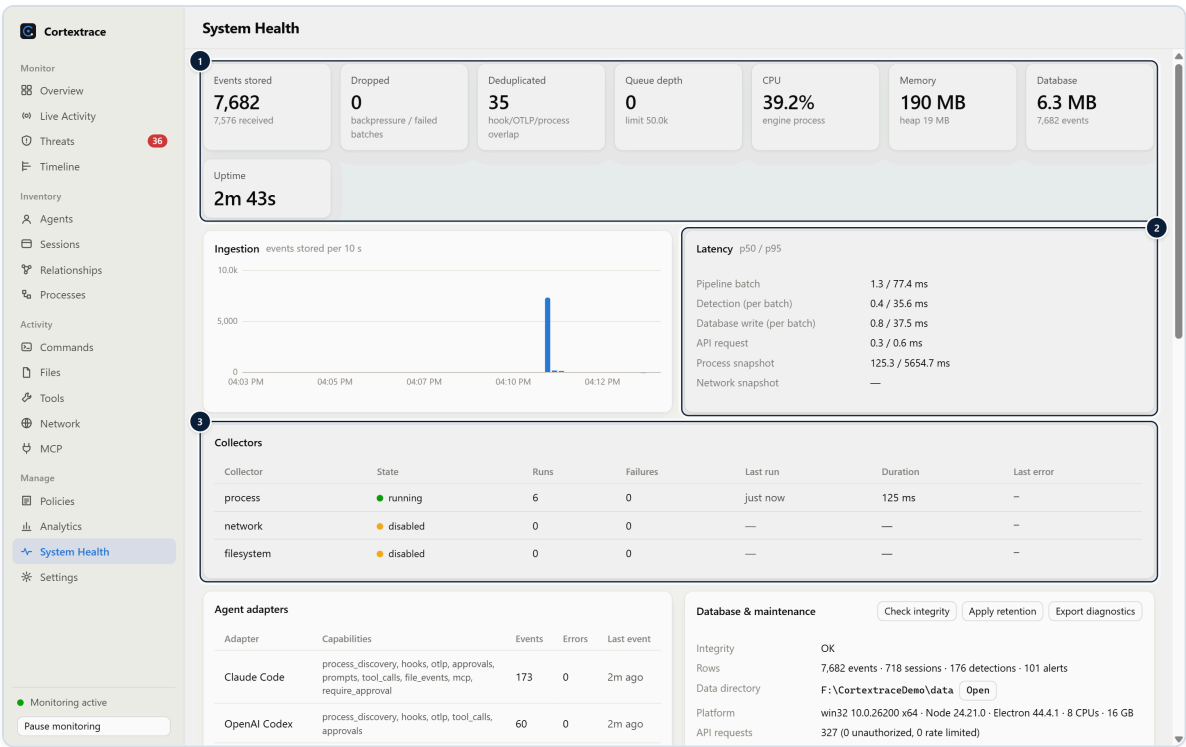


Figure 36 System Health: pipeline counters, latency percentiles, collectors, adapters, database and diagnostic log.

- 1

**Pipeline counters** — Stored, dropped and de-duplicated events, queue depth, CPU, memory, database size and uptime.
- 2

**Latency** — p50/p95 for pipeline, detection, storage, API and each collector.
- 3

**Collectors** — State, runs, failures, last run, duration and last error for each collector.

## 17 Extensibility

Adding support for an agent means implementing or configuring an adapter — not changing the pipeline.

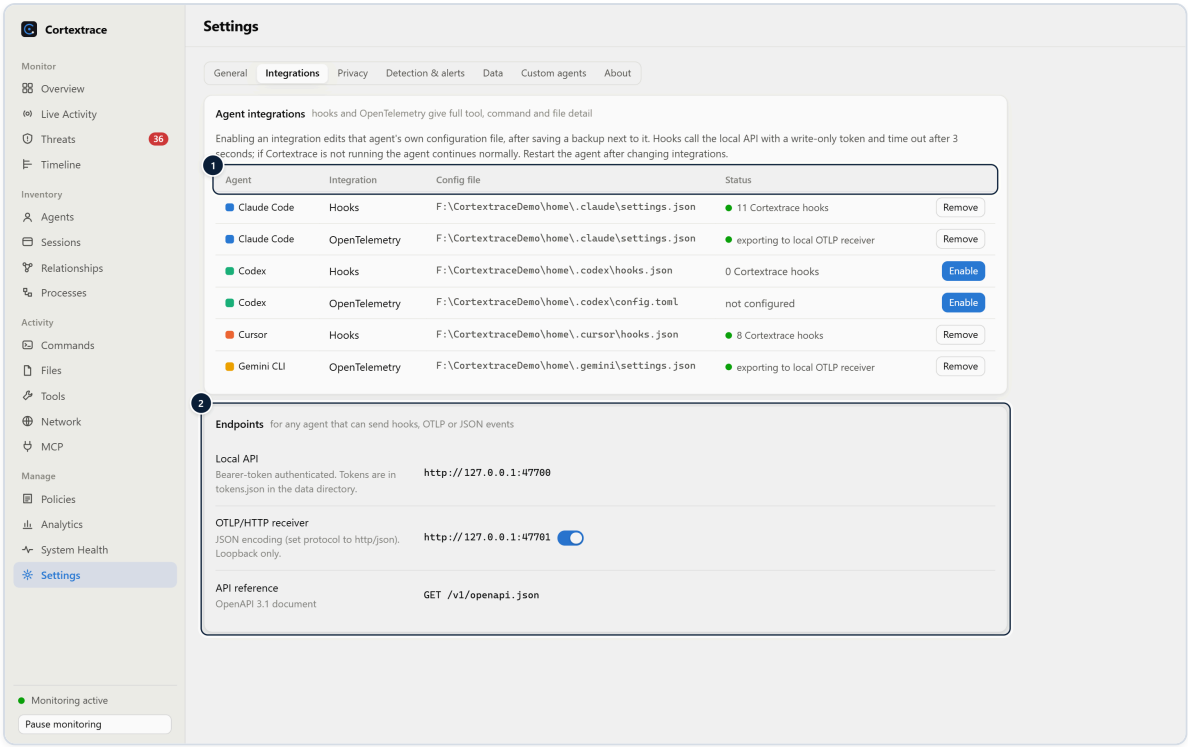
| Extension path                  | Effort                                            | Provides                                                                                                              |
|---------------------------------|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Custom agent (configuration)    | JSON in Settings, no code                         | Process discovery, a hook endpoint with event-name mapping, per-agent policies                                        |
| Local REST API or OpenTelemetry | A few lines in the agent                          | Full structured events from inside the agent; all detection and risk features apply                                   |
| Built-in adapter (code)         | Implement the <code>AgentAdapter</code> interface | Identity detection, hook normalisation, approval responses, integration installer, version and capability declaration |
| Detection rules and policies    | Rule with examples, or policy JSON                | New detections, automatically tested (rules) or configured per user (policies)                                        |

### Sending events from a custom agent

```
POST http://127.0.0.1:47631/v1/events
Authorization: Bearer <ingest token from tokens.json>
Content-Type: application/json

[
  { "event_type": "session_started", "agent_type": "research-bot", "session_id": "s1",
    "workspace": "/work/repo" },
  { "event_type": "command_execution", "agent_type": "research-bot", "session_id": "s1",
    "duration_ms": 120, "payload": { "command": "git status" } }
]
```

The API is documented by an OpenAPI 3.1 document at `GET /v1/openapi.json`. OpenTelemetry data can be sent as OTLP/HTTP JSON to `http://127.0.0.1:4318/v1/logs`, `/v1/traces` and `/v1/metrics`.



**Figure 37** Integrations: native hooks and telemetry per agent, with local endpoints for any compatible agent.

- 1 Integrations** — Hook and OpenTelemetry integrations per agent with config file and live status; enable/remove with automatic backup.
- 2 Local endpoints** — Loopback API (token-authenticated) and OTLP/HTTP JSON receiver for any compatible agent.

Plugin model

In this release, extensions are adapters compiled into the application or custom agents defined in configuration. Loading third-party adapter code at runtime is not supported.

# 18 Installation and Getting Started

---

## System requirements

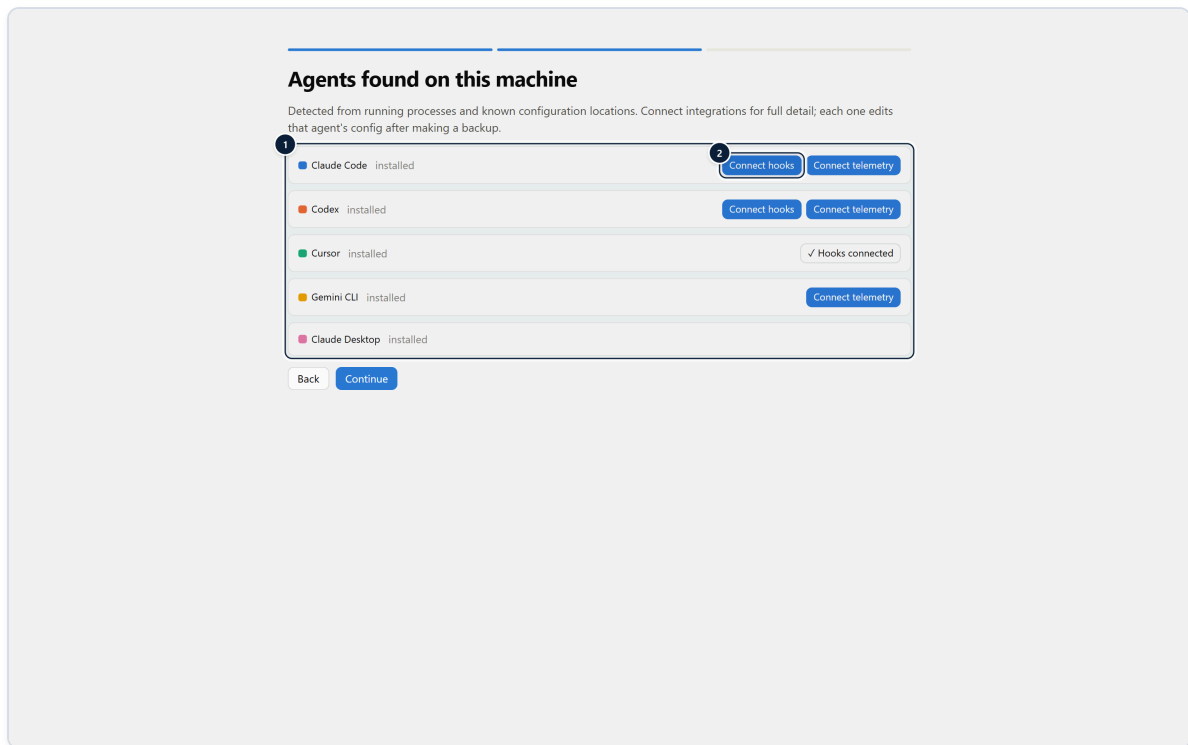
| Platform    | Requirement                                                                               |
|-------------|-------------------------------------------------------------------------------------------|
| Windows     | Windows 10 or 11, x64 (NSIS installer also built for arm64 by configuration)              |
| macOS       | DMG for Apple silicon and Intel, universal PKG (build targets configured)                 |
| Linux       | Applimage, DEB (x64, arm64), RPM (x64); a StatusNotifier host is needed for the tray icon |
| From source | Node.js 24 or later                                                                       |

## Installation

Installers are published on the [GitHub Releases page](#). The 0.1.0 release provides the Windows NSIS installer and MSI package built and install-tested for this document; macOS and Linux packages are produced by the repository's release workflow once it has been run on those platforms.

1. **Windows:** run `Cortextrace-0.1.0-win-x64.exe` (per-user installation, no administrator rights) or deploy `Cortextrace-0.1.0-win-x64.msi`. Silent installation: `Cortextrace-0.1.0-win-x64.exe /S`. Release builds without a code-signing certificate trigger a SmartScreen prompt.
2. **macOS:** open the DMG and drag Cortextrace to Applications, or run the PKG.
3. **Linux:** run the Applimage, or install the DEB / RPM package with the system package manager.
4. **From source:** `npm ci`, `npx electron scripts/icons.cjs`, `npm run build`, `npm start`. A headless engine is available with `npm run engine`.

## First launch



**Figure 38** The first-run guide lists agents found on the machine and offers one-click integrations.

- 1 **Agents found from config locations** — Agents detected from known installation/configuration paths on this machine (demo profile).
- 2 **One-click integration:** writes the agent's native hook or telemetry configuration after creating a backup.

1. Review the disclosure of data sources.
2. Connect hooks or telemetry for detected agents (optional; restart the agent afterwards).
3. Choose whether to start at login, then open the dashboard.

## Initial configuration

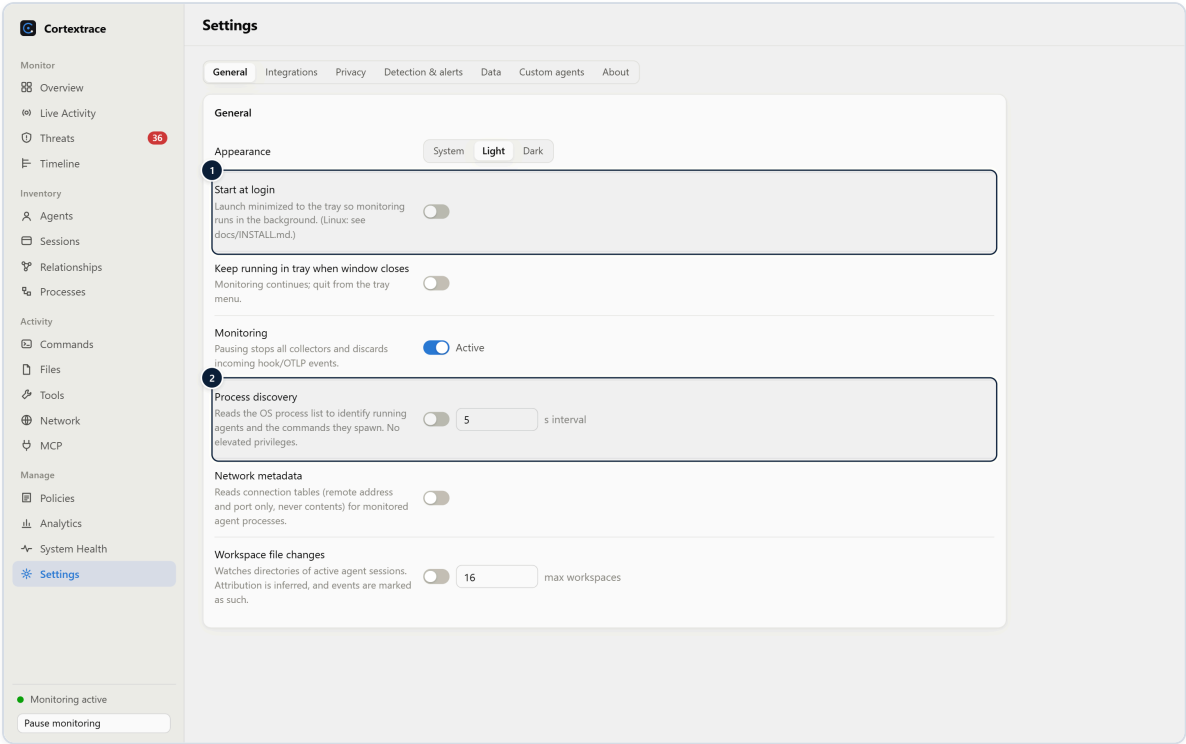


Figure 39 General settings: background operation and collectors.

- 1

**Background operation** — Start at login and keep running in the tray.
- 2

**Collectors** — Each collector can be disabled; interval and workspace limits are configurable.

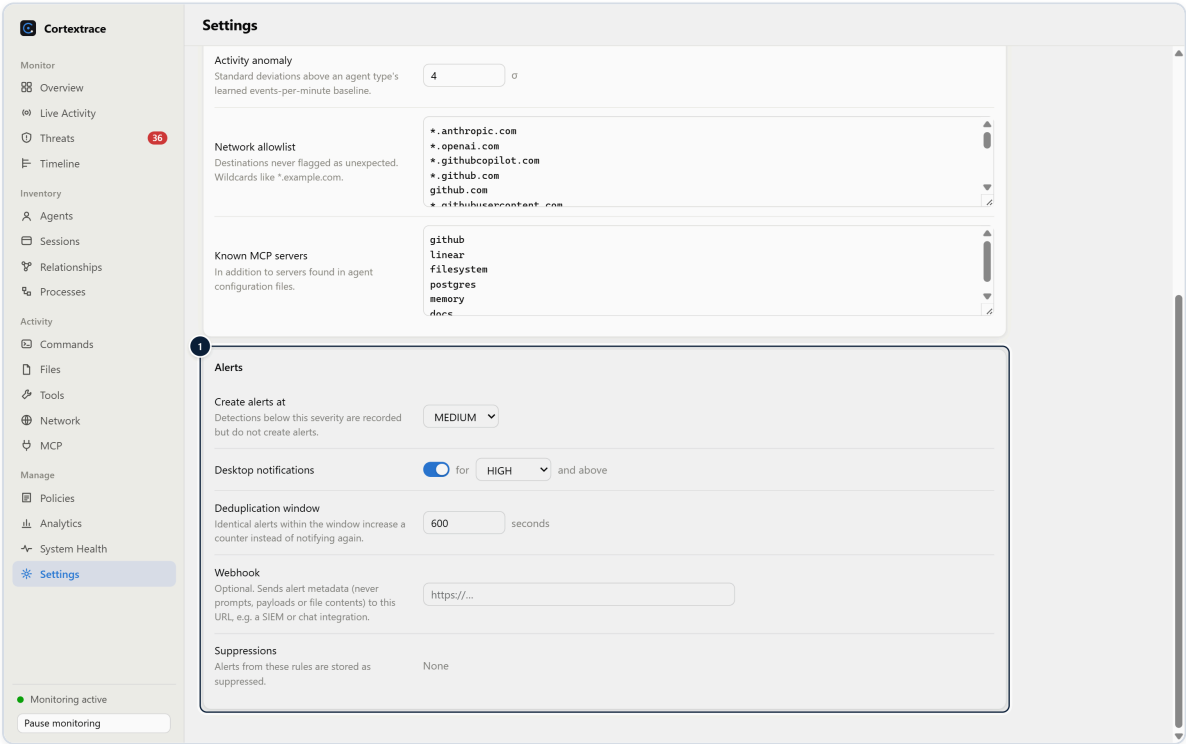


Figure 40 Alert routing: thresholds, notifications, de-duplication, webhook and suppressions.

- 1

**Alert routing** — Alert threshold, desktop notification threshold, de-duplication window, optional webhook and suppressions.

## Basic workflow

1. Work with agents as usual; Cortextrace records sessions in the background.
2. Check the Threats badge or desktop notifications for new alerts.
3. Open an incident, review the evidence and the session timeline, then acknowledge or suppress.
4. Refine policies, allowlists and privacy modes as needed.

# 19 Supported Platforms and Capabilities

| Capability                       | Windows                                         | macOS                             | Linux                                              |
|----------------------------------|-------------------------------------------------|-----------------------------------|----------------------------------------------------|
| Desktop app, tray, notifications | Validated                                       | Implemented                       | Implemented                                        |
| Process discovery                | Validated (PowerShell / CIM)                    | Implemented ( <code>ps</code> )   | Implemented ( <code>/proc</code> )                 |
| Network metadata                 | Validated ( <code>Get-NetTCPConnection</code> ) | Implemented ( <code>lsof</code> ) | Implemented ( <code>/proc/net/tcp</code> )         |
| Process working directory        | Not available                                   | Implemented ( <code>lsof</code> ) | Implemented ( <code>/proc/&lt;pid&gt;/cwd</code> ) |
| Workspace file watching          | Implemented                                     | Implemented                       | Implemented (subject to inotify limits)            |
| Start at login                   | Implemented                                     | Implemented                       | Manual autostart entry (documented)                |
| Installers                       | NSIS and MSI built and install-tested           | DMG / PKG configured              | ApplImage / DEB / RPM configured                   |

**Validated** means exercised on real hardware for this release: the Windows installer was installed silently, the installed application passed the automated end-to-end test across all dashboard pages, and it uninstalled cleanly. **Implemented** means the code and packaging configuration exist and unit tests cover the platform-independent logic, but the capability has not yet been run on that operating system for this release; continuous-integration workflows for macOS and Linux are included in the repository.

## Platform-specific limitations

Windows does not expose other processes' working directories without debug privileges. macOS may prompt for access when watching workspaces in protected folders. Linux file watching on very large repositories may require raising the inotify watch limit. Code signing and notarization require project-owned certificates that are not part of this release.

## 20 Security and Threat Model

### Security boundaries

- **Trust boundary:** the logged-in user session. Agents, the engine, the database and the dashboard all run as that user.
- **Network boundary:** the API and OTLP receiver listen only on the loopback interface; web pages in a browser are refused.
- **Renderer boundary:** the dashboard runs sandboxed and reaches the engine only through validated IPC channels.
- **Export boundary:** data leaves the device only through a user-configured webhook or OTLP endpoint.

### Threat categories considered

| Threat                                              | Mitigation                                                                                                   | Residual risk                                                            |
|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Remote attacker reaching the API                    | Loopback binding                                                                                             | None known                                                               |
| Malicious web page (CSRF, DNS rebinding)            | Origin and Host checks, JSON-only bodies, bearer tokens                                                      | None known for the token-protected API                                   |
| Other local processes as the same user              | Owner-only file permissions (POSIX), write-only ingest token, configuration changes logged                   | Out of scope: same-user code can forge telemetry or stop the application |
| Agent editing its own hooks or settings             | Agent control-file rule and self-configuration policy; integration status in Settings                        | Edits outside observed paths or before start are not seen                |
| Secrets leaking into Cortextrace data               | Redaction before storage, metadata-only prompts by default, no file contents                                 | Unrecognised secret formats; custom patterns available                   |
| Crafted payloads (injection, denial of service)     | Schema validation, size caps, rate limiting, bounded queue, parameterised SQL, escaped rendering, strict CSP | A local process can still send traffic up to the rate limit              |
| Detection evasion by a deliberately malicious agent | Multiple independent sources, obfuscation and encoding rules, hooks capture commands before execution        | Polling gaps; no kernel-level telemetry                                  |
| Misuse as surveillance software                     | Always visible, full disclosure at first run, pause control, no stealth mode, no remote upload by default    | Exports configured by a local administrator                              |

### Detection philosophy

Cortextrace separates **telemetry** (what was observed, with its fidelity), **detections** (rule, policy, heuristic or anomaly matches with confidence) and **policy violations** (breaches of rules the user configured). Suspicious indicators require human judgement; Cortextrace does not claim malware detection or classify intent.

## False-positive considerations

- Several rules are observations by design (for example package installation and CI/infrastructure edits).
- Trusted destinations and MCP servers can be added to allowlists; individual rules can be disabled; alerts can be suppressed per rule and agent.
- Prompt jailbreak phrasing is intentionally LOW severity with 40% confidence, because users often quote such text legitimately.

# 21 Product Release Summary

## Implemented capabilities

- Desktop application with tray, notifications, start at login, pause, first-run onboarding and light and dark themes.
- Discovery of 23 built-in agent types, dynamic LLM-client discovery and configurable custom agents.
- Collectors for process trees, network connection metadata and workspace file changes, with supervision and backoff.
- One-click integrations for Claude Code (hooks, OpenTelemetry), Codex (hooks, OpenTelemetry), Cursor (hooks) and Gemini CLI (OpenTelemetry).
- OTLP/HTTP JSON receiver and token-authenticated REST API with an OpenAPI 3.1 document; headless engine.
- Versioned event schema with 27 event types, redaction, privacy modes, de-duplication across sources.
- SQLite storage with retention, integrity checks and automatic recovery; JSONL and CSV export; optional OTLP export.
- 39 tested detection rules, policies with approval responses, heuristics, anomaly baselines and first-seen detection.
- Explainable risk scoring across 10 dimensions; alerts with de-duplication, incidents and suppressions.
- Seventeen dashboard pages including session timelines with replay, a relationship graph, analytics and system health.
- Automated tests (81 passing), desktop end-to-end test, performance benchmark, CI and release workflows, license audit.

## Known limitations

| Area                | Limitation                                                                                                                                                                                                            |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Platforms           | macOS and Linux builds and collectors are implemented but not yet validated on those systems for this release                                                                                                         |
| Signing and updates | Installers are unsigned in this release; there is no automatic update mechanism (planned after signing)                                                                                                               |
| Collection          | Process and network collection is polling-based; short-lived processes may be missed without hook integrations; network data is metadata only                                                                         |
| Attribution         | File changes are attributed by workspace (inferred); Windows provides no working directory for process-only sessions                                                                                                  |
| Integrations        | Hook and OpenTelemetry mappings for Cursor, Codex and Gemini CLI follow their documented formats and are covered by tests with representative payloads; this release did not exercise them against live installations |
| Storage             | Database is not encrypted at rest; overview aggregates slow down beyond roughly 200,000 events in the selected range (cached for 10 s)                                                                                |

| Area              | Limitation                                                                                          |
|-------------------|-----------------------------------------------------------------------------------------------------|
| Extensibility     | No runtime loading of third-party adapter code                                                      |
| Agent detail      | Capabilities and install paths are shown only for agents also found in known installation locations |
| Tamper resistance | Not designed to resist an attacker with code execution as the same user                             |

## Roadmap

- Validate and publish signed builds for macOS (notarized) and Linux; Authenticode signing for Windows.
- Opt-in automatic updates with signature verification.
- Optional encrypted storage.
- Opt-in enhanced collection using OS event tracing facilities behind an explicit permission step.
- Declarative multi-step correlation rules and broader rule taxonomy references.
- Additional native integrations and verified payload contracts for more agents.
- Full-text search for very large histories and forwarding templates for common SIEM products.

## 22 Company and Contact

Cortextrace is created and maintained by Manjunatha M at AI and ML consultants.

| Field                  | Value                                                                                                                 |
|------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Company                | AI and ML consultants                                                                                                 |
| Founder and maintainer | Manjunatha M                                                                                                          |
| Website                | <a href="https://aiandmlconsultants.com/">https://aiandmlconsultants.com/</a>                                         |
| Email                  | <a href="mailto:Manjunatha@aiandmlconsultants.com">Manjunatha@aiandmlconsultants.com</a>                              |
| GitHub repository      | <a href="https://github.com/manjunatha552355-rgb/cortextrace">https://github.com/manjunatha552355-rgb/cortextrace</a> |

### Open-source project

| Item                        | Detail                                                                                                                                                                                                                                                             |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| License                     | Apache License 2.0 ( <code>LICENSE</code> in the repository); third-party components listed in <code>THIRD_PARTY_NOTICES.md</code>                                                                                                                                 |
| Source code                 | <a href="https://github.com/manjunatha552355-rgb/cortextrace">https://github.com/manjunatha552355-rgb/cortextrace</a>                                                                                                                                              |
| Releases and installers     | <a href="https://github.com/manjunatha552355-rgb/cortextrace/releases">https://github.com/manjunatha552355-rgb/cortextrace/releases</a>                                                                                                                            |
| Issues and feature requests | <a href="https://github.com/manjunatha552355-rgb/cortextrace/issues">https://github.com/manjunatha552355-rgb/cortextrace/issues</a>                                                                                                                                |
| Contributing                | <code>CONTRIBUTING.md</code> — development setup, test commands, rule and adapter checklists                                                                                                                                                                       |
| Security reporting          | <code>SECURITY.md</code> — private reporting through GitHub Security Advisories on the repository, or by email to <a href="mailto:Manjunatha@aiandmlconsultants.com">Manjunatha@aiandmlconsultants.com</a> . Please do not open public issues for vulnerabilities. |
| Code of conduct             | <code>CODE_OF_CONDUCT.md</code>                                                                                                                                                                                                                                    |
| Documentation               | <code>docs/</code> — architecture, event schema, detection and risk, privacy, threat model, installation, adapters, design decisions, releasing                                                                                                                    |